

Pseudorandom Correlation Generators for Multiparty Beaver Triples over $\mathbb{F}_2$

Peihan Miao¹, Alice Murphy², Akshayaram Srinivasan³, and Max
Tromanhauser⁴

¹ Brown University

`peihan_miao@brown.edu`

² Oregon State University

`murphali@oregonstate.edu`

³ University of Toronto

`akshayaram@cs.toronto.edu`

⁴ Cornell University

`mft55@cornell.edu`

Abstract. We construct an efficient pseudorandom correlation generator (PCG) (Boyle et al., Crypto’19) for two-party *programmable oblivious linear evaluation (OLE)* functionality over $\mathbb{F}_2$. Our construction (i) has an efficient seed expansion phase, and (ii) comes with a concretely efficient protocol for distributing the seeds that makes black-box use of cryptography and runs in a constant number of rounds.

PCGs for programmable OLE are known to imply PCGs for generating n -party Beaver triples over $\mathbb{F}_2$. The resultant PCG has a seed setup phase whose communication cost is $n(n - 1)$ times than that of the programmable OLE protocol. The per-party seed size and the seed expansion time have a multiplicative overhead of $2(n - 1)$. Prior constructions for efficiently generating multiparty Beaver triples only worked for finite fields $\mathbb{F}_q$ where $q \geq 3$ or required one bit of per-party communication for each triple generated (and hence, do not satisfy the PCG definition). Thus, ours is the first concretely efficient PCG for generating Beaver triples over $\mathbb{F}_2$ in the multiparty setting.

Our distributed seed generation protocol generates $N = 2^{30}$ two-party programmable OLEs in 3.5 minutes with 255 MB of communication over a LAN network. The PCG seed size is around 55 MB and the expansion phase requires 10 PRG calls and around 229 thousand XOR and AND operations per triple, producing roughly 31,000 triples per second.

Our PCG for generating multiparty Beaver triples has lower concrete communication cost than the state-of-the-art for small number of parties. When compared to the FOLEAGE protocol (Bombar et al, Asiacrypt 2024) which requires one bit of per-party communication per triple that is generated, our communication cost is lower by $2.4\times$ when generating $N = 2^{36}$ triples between three parties and is $1.2\times$ lower for the case of five parties.

At a conceptual level, our protocol deviates from the prior approaches which relied on variants of dual learning parity with noise (LPN) assumption. Instead, our construction combines both the primal and dual versions of LPN to achieve the aforementioned efficiency.

Keywords: Pseudorandom Correlation Generators, Beaver Triples, Learning Parity with Noise

1 Introduction

Correlated randomness is an important resource in cryptography that enables modular and efficient constructions of secure multiparty computation (MPC) [31, 50]. Protocols using this paradigm involve an offline phase where the correlated randomness is generated. This phase is run before the private inputs and the function to be computed are known to the parties. The generated correlations in the offline phase are consumed in the online phase which is run after the inputs and the function to be computed are known. Such protocols typically have an extremely efficient online phase which is either information-theoretic or uses lightweight symmetric cryptographic tools. Starting with the seminal work of Beaver [4], this approach has had tremendous success in designing concretely efficient protocols in the two-party and multiparty settings [21, 33–35, 48, 49].

Traditional approaches to generate correlated randomness involved the parties communicating $\Omega(N)$ bits where N is the size of correlated randomness. However, this changed with the breakthrough work of Boyle et al. [10, 13] who gave a new approach that breaks the linear communication barrier. They introduced pseudorandom correlation generators (PCG) which allow generation of correlated randomness of size N with communication $o(N)$ bits. In a bit more detail, PCGs involve a seed setup phase where the parties are given correlated seeds. This seed setup phase is implemented by a secure computation protocol. The communication cost needed to run this protocol is $o(N)$ bits. After the seed setup, the parties locally expand these correlated seeds to obtain correlated randomness of size N . This seed expansion phase does not involve any communication between parties and hence, is *silent*. By now, we know concretely efficient PCGs for generating random oblivious transfer (OT) and vector oblivious linear evaluation (VOLE) correlations [10, 11, 13, 14, 20, 43, 46].

Multiparty Beaver Triples. An important class of correlations in the n -party setting for $n > 2$ is multiparty Beaver triples over $\mathbb{F}_2$. It consists of N triples of bits given to each party. Each triple in the sequence is generated by first sampling a_i, b_i uniformly from $\{0, 1\}$ for each $i \in [n]$. Let $c := (\sum_i a_i) \cdot (\sum_i b_i)$. Let $c_1, \dots, c_n$ be an additive secret sharing of c . The i -th party receives (a_i, b_i, c_i) .

Such multiparty Beaver triples can be used to construct secure protocols for computing Boolean circuits with an efficient information-theoretic online protocol which runs in $O(d)$ rounds (where d is the depth of the circuit to be computed) [4, 31], or can be used to construct concretely efficient constant-round protocols by assuming additional cryptographic tools [7, 32]. It is *crucial* in these protocols that the *triples are generated over $\mathbb{F}_2$* rather than over some other finite field.

PCGs for Multiparty Beaver Triples. Boyle et al. [14] gave a PCG for generating n -party Beaver triples over large finite fields (rather than $\mathbb{F}_2$). They

estimated that the seed expansion phase takes $2(n-1) \cdot 10$ seconds to generate $N = 10^6$ triples. In a follow-up work, Boyle et al. [11] gave a PCG for computing triples over $\mathbb{F}_2$. However, the seed expansion phase required more than 2 million PRG calls to compute a single triple (even in the two-party setting) and the distributed seed generation phase required non-black-box use of cryptography. Later, Bombar et al. [8] improved this to give an efficient PCG for $\mathbb{F}_q$ for any $q \geq 3$. However, they left the analysis of concrete efficiency for future work. A recent work of Bombar et al. [7] gave a concretely efficient PCG over $\mathbb{F}_4$ and used this to construct an efficient protocol for generating triples over $\mathbb{F}_2$. However, the protocol for $\mathbb{F}_2$ involves a broadcast of $N \cdot n$ bits and hence, is not a PCG.

Given the state-of-the-art, the following question is still open:

*Can we design a concretely efficient PCG for generating multiparty Beaver triples over $\mathbb{F}_2$ with an efficient distributed seed generation?*⁵

1.1 Our Results

We answer the above question in the affirmative by giving an efficient pseudorandom correlation generator for multiparty Beaver triples over $\mathbb{F}_2$ with an efficient distributed seed generation protocol. We achieve this by first giving an efficient construction of a PCG for a two-party functionality called *programmable oblivious linear evaluation (OLE)* over $\mathbb{F}_2$. This is known to imply a PCG for n -party Beaver triples with $2(n-1)$ multiplicative overhead in the per-party seed size and seed expansion time, and $n(n-1)$ overhead in the total communication cost of the seed setup phase. Our protocol for distributing the seeds for this PCG makes black-box use of cryptography.

We implement our PCG and distributed seed generation protocol and show concrete efficiency. Specifically, to generate $N = 2^{30}$ triples, the offline seed generation protocol requires 255 MB of communication and its runtime is 3.5 minutes over a LAN network. The generated PCG seed size is around 55 MB. The seed expansion phase requires (amortized) 10 PRG calls and 229 thousand XOR and AND operations per triple, producing around 31,000 triples per second.

The security of our PCG relies on both the primal and dual variants of learning parity with noise (LPN) [6] with regular error distribution. The optimized version that gives the above reported numbers relies on sparse primal LPN [2]. The dual LPN matrix is randomly sampled without any specific structure. The offline protocol additionally requires an additively homomorphic private-key encryption with function privacy.

Comparison with [7]. Recently, Bombar et al. [7] proposed a concretely efficient protocol for generating multiparty Beaver triples over $\mathbb{F}_2$. This protocol is not a PCG as it requires a per-party broadcast of one bit per triple. But the computation and communication cost of this protocol is concretely efficient and

⁵ We note that a concurrent and independent work by Li et al. [36] gave a PCG construction for generating multiparty Beaver triples over $\mathbb{F}_2$, which we discuss later.

is better than all other approaches for generating multiparty Beaver triples over $\mathbb{F}_2$.

Our protocol has better communication cost than the protocol in [7] when considering small number of parties. Specifically, to generate $N = 2^{36}$ Beaver triples, the (estimated) communication cost of our protocol is $2.4\times$ lower for the case of three parties and $1.2\times$ lower for the case of five parties. These savings would become substantial if we were to generate more triples. We remark that when considering a large number of parties, the communication cost of Bombar et al.’s work is better than our current cost. However, since our seed expansion phase is truly silent, the concrete communication cost of our protocol could be improved by developing a better distributed seed setup phase. We leave this problem open for future work.

Comparison with [36]. In a concurrent and independent work, Li et al. [36] gave a PCG construction for generating programmable OLE over $\mathbb{F}_2$. We give the comparison between their work and ours.

Their construction builds on QA-SD/Ring-LPN assumptions and crucially relies on the algebraic structure of polynomial rings in the underlying assumptions to get both correctness and the required efficiency, similar to prior works [7, 8, 14]. In contrast, our construction builds directly on LPN assumptions where the code matrices are random sparse matrices and hence, unstructured. This makes our assumptions arguably more standard.

We compare the concrete efficiency of their construction with ours below.

- **Seed generation:** We would like to note that they did not implement the seed generation protocol and hence, it is difficult to conduct a comprehensive comparison in terms of communication cost and runtime for the seed generation phase. However, they provide one estimate (in Section 9.3) for $c = 3, t = 27, n = 16$ and $\mathbb{F}_4$, which generates $2 \cdot 3^{16} (= 2^{26.35})$ OLEs over $\mathbb{F}_2$. We note that under these parameters, it only achieves 80-bit security (see their Table 6). Their estimated communication for seed generation is 26 MB. We estimate our communication cost for the seed generation phase for 2^{27} OLEs with 80-bit security to be 20.7 MB. This is 20% lower than theirs.
- **Seed size and expansion rate:** We achieve smaller seeds and higher expansion rates for certain choice of parameters. For instance, for 2^{28} OLEs with 128-bit security, their seed expansion rate is 2.04 while ours is 4.57 on P_0 ($2.2\times$ higher than theirs) and 3.26 on P_1 ($1.6\times$ higher). For 2^{30} OLEs with 128-bit security, their seed expansion rate is between $2.83 - 5.79$ while ours is 8.53 on P_0 ($1.5 - 3.0\times$ higher) and 6.24 on P_1 ($1.1 - 2.2\times$ higher).
- **Expansion time:** Their expansion time is only two times slower than FOLEAGE [7] and is faster than ours.

To summarize, our communication cost and seed size (in certain parameter regimes) are better than theirs while their seed expansion time is faster than ours. Further, our work relies on a more standard assumption that is devoid of any algebraic structure.

Recent attacks on QA-SD. Recent work by Bouillaguet et al. [9] proposes new attacks on the Quasi-Abelian Syndrome Decoding (QA-SD) assumption used in [7] and [36]. Specifically, for the above parameters $c = 3, t = 27, n = 16$ and $\mathbb{F}_4$ (as chosen in both works [7, 36]), the QA-SD assumption is vulnerable to the new attack, with success probability as high as 61%. In contrast, our work relies on more standard LPN assumptions.

2 Technical Overview

In this section, we give a high-level overview of the techniques used in our construction of a pseudorandom correlation generator (PCG) for multiparty Beaver triples. As in the previous works [7, 8, 13, 14], we start with a useful two-party abstraction known as programmable oblivious linear evaluation (OLE) functionality and use this to generate multiparty Beaver triples.

Programmable OLE. A functionality for programmable OLE is parameterized by a pseudorandom generator (PRG) G . The parties P_0 and P_1 send seeds r_0 and r_1 to the ideal functionality. The functionality computes $G(r_0)$ and $G(r_1)$ and outputs a random additive secret sharing of the point-wise product given by $G(r_0) \odot G(r_1)$.

It is known that a PCG for programmable OLE functionality implies a PCG for multiparty Beaver triples [13]. To see why this is the case, consider a protocol where each party P_i samples two PRG seeds $(s_{i,L}, s_{i,R})$. Every ordered pair of parties (P_i, P_j) make a call to the programmable OLE functionality with inputs being the seeds $s_{i,L}$ and $s_{j,R}$ respectively. The party P_i sets $\mathbf{a}_i = G(s_{i,L})$, $\mathbf{b}_i = G(s_{i,R})$ and $\mathbf{c}_i$ to be $\mathbf{a}_i \odot \mathbf{b}_i$ plus the sum of all outputs from the programmable OLE functionality for every call involving P_i . It can be verified that $(\sum_i \mathbf{a}_i) \odot (\sum_i \mathbf{b}_i) = \sum_i \mathbf{c}_i$. See a formal description of this protocol in Appendix A. This protocol makes $n(n-1)$ calls to the programmable OLE functionality. Hence, the per-party seed size and the computation cost for the seed expansion phase grows by a multiplicative factor of $2(n-1)$. The communication cost of the seed setup phase is $n(n-1)$ times that of the programmable OLE protocol.

PCG for programmable OLE. We give a new construction of a PCG for generating programmable OLE. In almost all the previous works, the PRG G was constructed using (variants) of dual learning parity with noise (LPN) assumption. These approaches lead to a significant computational cost when generating triples over $\mathbb{F}_2$. However, in our construction, the PRG G is constructed with the primal version of LPN.

Recall that we can view the primal version of the LPN as a PRG where the seed is given by the LPN secret $\mathbf{s} \in \{0, 1\}^n$ and a sparse error vector $\mathbf{e} \in \{0, 1\}^N$. The output of the PRG is $\mathbf{A} \cdot \mathbf{s} + \mathbf{e}$ where $\mathbf{A}$ is a randomly chosen public matrix in $\{0, 1\}^{N \times n}$.

Given the above instantiation of the PRG, the task of constructing a PCG for programmable OLE reduces to constructing additive secret sharing of:

$$(\mathbf{A} \cdot \mathbf{s}_0 + \mathbf{e}_0) \odot (\mathbf{A} \cdot \mathbf{s}_1 + \mathbf{e}_1)$$

$$= \underbrace{(\mathbf{A} \cdot \mathbf{s}_0) \odot (\mathbf{A} \cdot \mathbf{s}_1)}_{\boxed{1}} + \underbrace{\mathbf{e}_0 \odot (\mathbf{A} \cdot \mathbf{s}_1)}_{\boxed{2}} + \underbrace{(\mathbf{A} \cdot \mathbf{s}_0) \odot \mathbf{e}_1}_{\boxed{3}} + \underbrace{\mathbf{e}_0 \odot \mathbf{e}_1}_{\boxed{4}},$$

where P_0 holds $\mathbf{s}_0, \mathbf{e}_0$ and P_1 holds $\mathbf{s}_1, \mathbf{e}_1$. Due to the linearity property of additive shares, it is sufficient to construct PCGs for each of the above four terms.

Generating Additive Sharing of $\boxed{1}$. The first term is given by

$$(\mathbf{A} \cdot \mathbf{s}_0) \odot (\mathbf{A} \cdot \mathbf{s}_1).$$

The i -th entry in the above column vector is given by $\langle \mathbf{a}_i, \mathbf{s}_0 \rangle \cdot \langle \mathbf{a}_i, \mathbf{s}_1 \rangle = \langle \mathbf{a}_i \otimes \mathbf{a}_i, \mathbf{s}_0 \otimes \mathbf{s}_1 \rangle$, where $\mathbf{a}_i$ is the i -th row of $\mathbf{A}$. If we have a mechanism to generate an additive secret sharing of $\mathbf{s}_0 \otimes \mathbf{s}_1$, we would be essentially done as we can compute the inner product of the share with $\mathbf{a}_i \otimes \mathbf{a}_i$ to obtain additive secret sharing of term $\boxed{1}$. This is possible if we have an additively homomorphic encryption (AHE) scheme with function privacy. Specifically, P_0 sends an encryption of their LPN secret $\mathbf{s}_0$ to P_1 . P_1 homomorphically computes the tensor product and adds a random pad to the output and sends the ciphertext back to the P_0 . P_1 outputs the random pad as its share and P_0 decrypts the ciphertext and outputs the resultant plaintext as its share. Unfortunately, for the concrete parameter regimes where primal LPN is believed to have 128-bit of security, the size of the AHE ciphertext turns out to be prohibitively large.

To overcome this issue, we generate $\mathbf{s}_1$ as the output of the dual LPN PRG. Recall that the dual LPN PRG is parameterized by a randomly chosen compressing matrix $\mathbf{H}$ and the seed is a sparse error vector $\mathbf{e}$. The output of the PRG is given by $\mathbf{H} \cdot \mathbf{e}$. We set $\mathbf{s}_1 = \mathbf{H} \cdot \mathbf{e}$ where $\mathbf{e}$ is randomly chosen sparse error vector.

Thus, term $\boxed{1}$ now becomes:

$$\begin{aligned} (\mathbf{A} \cdot \mathbf{s}_0) \odot (\mathbf{A} \cdot \mathbf{s}_1) &= (\mathbf{A} \cdot \mathbf{s}_0) \odot (\mathbf{A} \cdot (\mathbf{H} \cdot \mathbf{e})) \\ &= (\mathbf{A} \cdot \mathbf{s}_0) \odot (\underbrace{(\mathbf{A} \cdot \mathbf{H})}_{:=\mathbf{B}} \cdot \mathbf{e}). \end{aligned}$$

The i -th entry of the above column vector is given by $\langle \mathbf{a}_i \otimes \mathbf{b}_i, \mathbf{s}_0 \otimes \mathbf{e} \rangle$. As before, the task reduces to giving an efficient protocol for computing an additive secret sharing of $\mathbf{s}_0 \otimes \mathbf{e}$.

Let $\mathbf{e}^\top = (e_1, \dots, e_k)$. We view $\mathbf{s}_0 \otimes \mathbf{e}$ as a $k \times n$ matrix $\mathbf{M}$ given by:

$$\mathbf{M} = \mathbf{s}_0 \otimes \mathbf{e} = \begin{pmatrix} e_1 \cdot \mathbf{s}_0^\top \\ e_2 \cdot \mathbf{s}_0^\top \\ \vdots \\ e_k \cdot \mathbf{s}_0^\top \end{pmatrix}. \quad (1)$$

Recall that the error vector $\mathbf{e}$ is sparse and hence, many rows of the matrix $\mathbf{M}$ are all zeroes. Thus, $\mathbf{M}$ has a short description. This allows us to construct an efficient protocol to generate an additive secret sharing of $\mathbf{M}$.

We give a high-level overview of the protocol. Assume that $\mathbf{e}$ comes from the regular error distribution. That is, the error vector can be sub-divided into several blocks and there is exactly one error position in each block. This error position is randomly chosen. This allows us to view $\mathbf{M}$ has a set of point functions where the function outputs the payload $\mathbf{s}_0$ at a single randomly chosen location. Thus, generating an additive secret sharing of $\mathbf{M}$ reduces to constructing a PCG for point functions where one party has the random location and the other party has the payload. For this purpose, we use the efficient protocol given by Boyle et al. [12].

We note that the main bottleneck in computing additive sharing of [1] is not computing additive shares of the matrix $\mathbf{M}$. Rather, the step that takes the maximum amount of time and memory is computing the inner product of $\mathbf{a}_i \otimes \mathbf{b}_i$ with the secret sharing of $\mathbf{s}_0 \otimes \mathbf{e}$ for each i . To speed up this computation, we make use of a sparse $\mathbf{a}_i$ and use sparse LPN [2] to argue security. This allows us to keep the seed expansion time from increasing too much.

Generating Additive Sharing of [2] & [3]. Note that terms [2] and [3] are symmetric and hence, we explain how to compute additive secret sharing of [2]. Recall that term [2] is given by:

$$\mathbf{e}_0 \odot (\mathbf{A} \cdot \mathbf{s}_1).$$

Let $\mathbf{e}_0^\top = (e_{0,1}, \dots, e_{0,N})$. The i -th entry in the above vector is given by $e_{0,i} \cdot \langle \mathbf{a}_i, \mathbf{s}_1 \rangle$. As before, we will be done if we get an additive secret sharing of the following $N \times n$ matrix $\mathbf{M}$ given by:

$$\mathbf{M} = \begin{pmatrix} e_{0,1} \cdot \mathbf{s}_1^\top \\ e_{0,2} \cdot \mathbf{s}_1^\top \\ \vdots \\ e_{0,N} \cdot \mathbf{s}_1^\top \end{pmatrix}. \quad (2)$$

This $\mathbf{M}$ has the same structure as the one in Equation 1. Unfortunately, the protocol for computing $\mathbf{M}$ in the previous case does not give concrete efficiency in this case. This is because $\mathbf{e}$ in Equation 1 comes from the dual LPN error distribution whereas $\mathbf{e}_0$ in Equation 2 comes from the primal LPN error distribution. The dual LPN assumption can tolerate error vectors which are much more sparse than the primal error vectors. Additionally, the communication complexity of Boyle et al.'s protocol [12] for computing additive secret sharing of point functions grows with the size of the payload. In this case, the payload is $\mathbf{s}_1$ and this blows up the communication cost significantly. Hence, the previous protocol does not give good concrete numbers in this case.

Approach-1: Use of Additively Homomorphic Encryption. To overcome this, we design another protocol that makes use of an additively homomorphic encryption scheme with function privacy. Assume that $\mathbf{e}_0$ again comes from the regular error distribution. For every index $i \in [m]$ such that $e_{0,i} = 1$, the parties first compute an additive secret sharing $\langle \mathbf{a}_i, \mathbf{s}_1 \rangle$. This can be achieved by P_1

sending an encryption of $\mathbf{s}_1$ using the AHE to P_0 . P_0 computes an encryption of $\langle \mathbf{a}_i, \mathbf{s}_1 \rangle$ for each i such that $e_{0,i} = 1$. P_0 homomorphically adds a random pad to the underlying messages and sends the resulting ciphertexts to P_1 . P_1 decrypts these ciphertexts and the parties now have an additive secret sharing of the required terms. Note that to prevent P_1 from learning the index i where $e_{0,i} = 1$, we need the AHE scheme to additionally satisfy function privacy.

Since $\mathbf{e}_0$ comes from the regular error distribution, we can sub-divide this $\mathbf{e}_0$ into several blocks where each block has exactly one error position. Computing the additive secret sharing of term $\boxed{2}$ now reduces to computing additive secret sharing of point functions, one for each block, with the payload in each block equal to $\langle \mathbf{a}_i, \mathbf{s}_1 \rangle$ where i is the non-zero index of $\mathbf{e}_0$ in that block. To achieve this, the parties use the protocol from [12]. Specifically, P_0 uses the error location i and P_1 uses its share of $\langle \mathbf{a}_i, \mathbf{s}_1 \rangle$ as inputs to the protocol. At the end of the protocol, P_0 and P_1 have an additive secret sharing of a point function that outputs P_1 's share at error location i . P_0 finally adds its share of $\langle \mathbf{a}_i, \mathbf{s}_1 \rangle$ at location i to obtain an additive secret sharing of a point function that outputs $\langle \mathbf{a}_i, \mathbf{s}_1 \rangle$ at location i .

When compared to the previous instantiation of the protocol where the payload size is $|\mathbf{s}_1|$ bits, the payload in the above instantiation is just a single bit. This allows us to reduce the number of bits of communication in the offline phase of distributed seed generation.

Approach-2: Using Learning with Errors. We give another direct approach to generate an additive secret sharing of $\langle \mathbf{a}_i, \mathbf{s}_1 \rangle$ based on the Learning with Errors (LWE) assumption [44]. This has lower asymptotic cost than the AHE-based solution. This solution builds on the trapdoor hash functions introduced by Döttling et al. [24].

Recall that the LWE assumption states that it is computationally hard to distinguish $(\mathbf{A}, \mathbf{A} \cdot \mathbf{s} + \mathbf{e})$ from $(\mathbf{A}, \mathbf{z})$ where $\mathbf{A} \leftarrow \mathbb{Z}_p^{n \times n'}$, $\mathbf{s} \leftarrow \mathbb{Z}_p^{n'}$, $\mathbf{e}$ is sampled from a distribution with low ℓ_∞ norm, and $\mathbf{z} \leftarrow \mathbb{Z}_p^n$.

To generate an additive secret sharing of $\langle \mathbf{a}_i, \mathbf{s}_1 \rangle$,

- P_1 samples an LWE secret $\mathbf{s}$, an error vector $\mathbf{e}$ and sends $\mathbf{A} \cdot \mathbf{s} + \mathbf{e} + \mathbf{s}_1 \cdot p/2$ to P_0 .⁶
- P_0 sends $\mathbf{a}_i^T \cdot \mathbf{A}$ to P_1 .
- P_0 computes $\mathbf{a}_i^T \cdot (\mathbf{A} \cdot \mathbf{s} + \mathbf{e} + \mathbf{s}_1 \cdot p/2)$ and P_1 computes $\mathbf{a}_i^T \cdot \mathbf{A} \cdot \mathbf{s}$.

Now, P_0 and P_1 hold a noisy secret sharing of $\langle \mathbf{a}_i, \mathbf{s}_1 \rangle \cdot p/2$ where the inner product is taken over $\mathbb{Z}_p$. To convert this noisy additive secret sharing to a noiseless additive sharing of $\langle \mathbf{a}_i, \mathbf{s}_1 \rangle$ over $\mathbb{F}_2$, we use the spooky rounding technique of Dodis et al. [23]. This rounding succeeds as long as the $\ell_\infty(\mathbf{a}_i^T \cdot \mathbf{e}) \ll p$. To additionally hide $\mathbf{a}_i$ from P_1 , we append a string of zeroes to $\mathbf{s}_1$ and append a random string to $\mathbf{a}_i$. This preserves the inner product between the two vectors. This also ensures that $\mathbf{a}_i^T \cdot \mathbf{A}$ is statistically close to random by the leftover hash lemma.

⁶ We can generate $\mathbf{A}$ as the output of a random oracle and hence, there is no need to explicitly send $\mathbf{A}$.

Remark. We adopted Approach-1 because based on our estimation, it achieves better concrete efficiency in terms of communication cost.

Generating Additive Sharing of [4]. This term is given by:

$$\mathbf{e}_0 \odot \mathbf{e}_1.$$

Assume that $\mathbf{e}_0$ and $\mathbf{e}_1$ come from regular error distribution. We can sub-divide this $\mathbf{e}_0$ into several blocks where each block has exactly one error position. Computing additive secret sharing of term [4] reduces to computing additive secret sharing of point functions, one for each block, with the payload is equal to the bit that indicates whether the error position in $\mathbf{e}_1$ in that block is same as the error position in $\mathbf{e}_0$ in that block. This reduces to generating secret sharing of equality testing, which can be done in a constant number of rounds using a garbled circuits based protocol. However, for concrete efficiency we use the protocol from [19]. We note that technically this is not a constant-round protocol. But for all practical parameters, the round complexity of this protocol turns out to be a small constant and hence, we use this to implement the equality testing functionality.

3 Preliminaries

3.1 Notation

Let $\lambda, \sigma \in \mathbb{N}$ denote the computational and statistical security parameters, respectively. The acronym PPT stands for probabilistic polynomial time. $\text{negl}(\lambda)$ denotes a negligible function in λ , meaning a function that vanishes faster than any inverse polynomial in λ .

Let $x := y$ denote the assignment of variable x to the value y . If $\mathcal{A}$ is a deterministic algorithm, $y := \mathcal{A}(x)$ denotes the assignment of variable y to the output of $\mathcal{A}(x)$. If $\mathcal{A}$ is randomized, $y \leftarrow \mathcal{A}(x)$ is used. The random coins, r , of $\mathcal{A}$ can be written explicitly in the function input, i.e., $y := \mathcal{A}(x; r)$. If S is a finite set, $x \leftarrow S$ denotes the experiment of sampling uniformly at random an element x from S . If D is a distribution over S , $x \leftarrow D$ denotes sampling x from S according to D .

Let D_0 and D_1 be distribution ensembles. D_0 is said to be statistically (resp. computationally) indistinguishable from D_1 , denoted by $D_0 \approx_s D_1$ (resp. $D_0 \approx_c D_1$) if no unbounded (resp. PPT) adversary can distinguish between the distributions except with probability at most $\text{negl}(\sigma)$ (resp. $\text{negl}(\lambda)$).

Vectors are denoted in boldface, for example, $\mathbf{x}$. Vector components are written as $\mathbf{x} = (x_1, \dots, x_n)$, where $|\mathbf{x}| = n$ is the length. Matrices are written in bold capital letters, for example, $\mathbf{A}$. For $n \in \mathbb{N}$, $[n]$ denotes the set $\{1, \dots, n\}$.

$\odot$ denotes the component-wise product of vectors or matrices. The dot product of vectors $\mathbf{x}$ and $\mathbf{y}$ is written as $\langle \mathbf{x}, \mathbf{y} \rangle$. For instance, let $\mathbf{x} = (x_1, \dots, x_n)$ and $\mathbf{y} = (y_1, \dots, y_n)$; then $\mathbf{x} \odot \mathbf{y} = (x_1 \cdot y_1, \dots, x_n \cdot y_n)$, and $\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{i \in [n]} x_i \cdot y_i$. Finally, let $[x]_b$ denote party P_b 's additive secret share of x .

3.2 Learning Parity with Noise

Code Generator Algorithm. Let $\mathbb{F}$ be a finite field and let $n(\cdot), m(\cdot)$ be polynomials. For ease of notation, we will use n and m to denote $n(\lambda)$ and $m(\lambda)$ respectively. A code generator algorithm C is a PPT algorithm that takes as input $(\mathbb{F}, n, m)$ and outputs a matrix $\mathbf{A} \in \mathbb{F}^{m \times n}$.

Definition 1 (Primal Learning Parity with Noise (LPN) Assumption [6]). *The primal Learning Parity with Noise (LPN) assumption with respect to a code generator algorithm C and error distribution $\chi(\mathbb{F}, n, m)$, states the following:*

$$\begin{aligned} \{(\mathbf{A}, \mathbf{y}) \mid \mathbf{A} \leftarrow C(\mathbb{F}, n, m), \mathbf{s} \leftarrow \mathbb{F}^n, \mathbf{e} \leftarrow \chi(\mathbb{F}, n, m), \mathbf{y} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e}\} \\ \approx_c \{(\mathbf{A}, \mathbf{y}) \mid \mathbf{A} \leftarrow C(\mathbb{F}, n, m), \mathbf{y} \leftarrow \mathbb{F}^m\}. \end{aligned}$$

The above definition captures various choices of code generator algorithms such as sparse LPN [2] as well as different error distributions.

In this work, we consider the regular noise distribution over bits. Let $\text{Reg}_t^m(\mathbb{F}_2)$ denote a regular noise distribution where sampled vectors are the concatenation of t random unit vectors of length m/t . So, vectors sampled from $\text{Reg}_t^m(\mathbb{F}_2)$ will have length m and Hamming weight t .

3.3 Dual Learning Parity with Noise

We also work with the dual version of the Learning Parity with Noise assumption. Here, we consider a dual code generator algorithm D that takes as input $(\mathbb{F}, n, m)$ and outputs a parity check matrix $\mathbf{H} \in \mathbb{F}^{(m-n) \times m}$.

Definition 2 (Dual Learning Parity with Noise (LPN) Assumption). *The dual Learning Parity with Noise (LPN) assumption with respect to a dual code generator algorithm D and error distribution $\chi(\mathbb{F}, n, m)$, states the following:*

$$\begin{aligned} \{(\mathbf{H}, \mathbf{y}) \mid \mathbf{H} \leftarrow D(\mathbb{F}, n, m), \mathbf{e} \leftarrow \chi(\mathbb{F}, n, m), \mathbf{y} = \mathbf{H} \cdot \mathbf{e}\} \\ \approx_c \{(\mathbf{H}, \mathbf{y}) \mid \mathbf{H} \leftarrow D(\mathbb{F}, n, m), \mathbf{y} \leftarrow \mathbb{F}^{m-n}\}. \end{aligned}$$

3.4 Additively Homomorphic Encryption (AHE)

Definition 3 (Additively Homomorphic Private-Key Encryption). *An additively homomorphic private-key encryption scheme with key space $\mathcal{K}$, message space $\mathcal{M}$, and ciphertext space $\mathcal{C}$, is given by the algorithms AHE.Gen , AHE.Enc , AHE.Dec , AHE.Eval where,*

- $\text{AHE.Gen}(1^\lambda)$, on input 1^λ , probabilistically outputs a key $\mathbf{k} \in \mathcal{K}$.
- $\text{AHE.Enc}(\mathbf{k}, x)$, on input a key $\mathbf{k} \in \mathcal{K}$ and a message $x \in \mathcal{M}$, probabilistically outputs a ciphertext $y \in \mathcal{C}$.

- **AHE.Dec**(k, y), on input a key $k \in \mathcal{K}$ and a ciphertext $y \in \mathcal{C}$, deterministically outputs $x \in \mathcal{M} \cup \{\perp\}$.
- **AHE.Eval**($\{y_1, \dots, y_n\}, f$), on input a set of ciphertexts $\{y_1, y_2, \dots, y_n\}$ and a function f , probabilistically outputs a new ciphertext $y \in \mathcal{C}$. We require that $f \in \mathcal{F}_n^{\text{AHE}}$, where $n \in \mathbb{N}$ and $\mathcal{F}_n^{\text{AHE}} : \mathcal{M}^{\times n} \rightarrow \mathcal{M}$ is the family of functions consisting only of additions of inputs and multiplication by scalars.

We require AHE to satisfy the following properties:

- **Correctness of Decryption:** For all $\lambda \in \mathbb{N}$, $k \in \mathcal{K}$, and $x \in \mathcal{M}$, we have that $\Pr[x = \text{AHE.Dec}(k, \text{AHE.Enc}(k, x))] = 1$.
- **Correctness of Homomorphic Evaluation:** For all $\lambda \in \mathbb{N}$, $f \in \mathcal{F}_n^{\text{AHE}}$, and $x_1, \dots, x_n \in \mathcal{M}$, we have that

$$\Pr[\text{AHE.Dec}(k, y) = f(x_1, \dots, x_n)] = 1$$

where $k \leftarrow \text{AHE.Gen}(1^\lambda)$, $y \leftarrow \text{AHE.Eval}(\{\text{AHE.Enc}(k, x_1), \dots, \text{AHE.Enc}(k, x_n)\}, f)$.

- **IND-CPA Security:** For all $\lambda \in \mathbb{N}$, $k \in \mathcal{K}$, and all pairs $x_0, x_1 \in \mathcal{M}$ we have $\text{AHE.Enc}(k, x_0) \approx_c \text{AHE.Enc}(k, x_1)$ under the chosen plaintext attack.
- **Function Privacy:** For all $\lambda \in \mathbb{N}$, $f \in \mathcal{F}_n^{\text{AHE}}$, and $x_1, \dots, x_n \in \mathcal{M}$, we have

$$\begin{aligned} & (k, (\text{AHE.Enc}(k, x_1), \dots, \text{AHE.Enc}(k, x_n)), \text{AHE.Eval}(\{\text{AHE.Enc}(k, x_1), \dots, \\ & \quad \text{AHE.Enc}(k, x_n)\}, f)) \\ & \approx_s (k, (\text{AHE.Enc}(k, x_1), \dots, \text{AHE.Enc}(k, x_n)), \text{AHE.Enc}(k, f(x_1, \dots, x_n))) \end{aligned}$$

where $k \leftarrow \text{AHE.Gen}(1^\lambda)$.

3.5 Secure Multiparty Computation

In this section, we recall the standard definition of secure multiparty computation against semi-honest adversaries, following the the universal composability (UC) framework [17].

Definition 4 (Semi-honest secure MPC). Let Π be a protocol computing an n -party functionality $\mathcal{F}$. We say that Π securely computes $\mathcal{F}$ against semi-honest adversaries if for every semi-honest adversary $\mathcal{A}$ corrupting an arbitrary subset $\mathcal{C} \subsetneq [n]$ of the parties, there exists a PPT simulator Sim such that for every choice of inputs $x_1, \dots, x_n$, we have:

$$\begin{aligned} & (\text{view}_{\mathcal{A}}(\Pi(x_1, \dots, x_n)), \{\text{out}_i\}_{i \in \mathcal{H}}) \\ & \approx_c (\text{Sim}(\{x_i\}_{i \in \mathcal{C}}, \{\mathcal{F}_i(x_1, \dots, x_n)\}_{i \in \mathcal{C}}, \{\mathcal{F}_i(x_1, \dots, x_n)\}_{i \in \mathcal{H}}) \end{aligned}$$

where $\mathcal{H} = [n] \setminus \mathcal{C}$ denotes the set of honest parties, $\text{view}_{\mathcal{A}}(\Pi(x_1, \dots, x_n))$ denotes the view of the adversary $\mathcal{A}$ during the execution of the protocol Π , out_i denotes the output of party i in the real world, and $\mathcal{F}_i(x_1, \dots, x_n)$ denotes the output of party i in the ideal world.

3.6 Puncturable Pseudorandom Functions

Definition 5 (Puncturable Pseudorandom Function (PPRF)). A puncturable pseudorandom function (PPRF) with key space $\mathcal{K}$, domain $\mathcal{X}$, and range $\mathcal{Y}$ is given by algorithms $(\text{PRF.Gen}, \text{PRF.Punct}, \text{PRF.Eval})$ where,

- $\text{PRF.Gen}(1^\lambda)$ takes as input 1^λ and outputs a key $K \in \mathcal{K}$.
- $\text{PRF.Punct}(K, S)$ takes as input a key $K \in \mathcal{K}$ and a set $S \subset \mathcal{X}$, and outputs a punctured key $K\{S\} \in \mathcal{K}$.
- $\text{PRF.Eval}(K\{S\}, x)$ takes as input a key $K\{S\} \in \mathcal{K}$ and $x \in \mathcal{X}$. If $x \notin S$, it outputs $\text{PRF.Eval}(K, x)$; else $\perp$ is output.

PPRFs must additionally satisfy the following properties.

- **Pseudorandomness without puncturing:** For all PPT adversaries $\mathcal{A}$, it holds that

$$\left| \Pr[\mathcal{A}^{\text{PRF.Eval}(K, \cdot)}(1^\lambda) = 1] - \Pr[\mathcal{A}^{f(\cdot)}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda),$$

where $K \leftarrow \text{PRF.Gen}(1^\lambda)$ and f is a random function from $\mathcal{X}$ to $\mathcal{Y}$.

- **Functionality under puncturing:** For all $\lambda \in \mathbb{N}$, polynomial-sized sets $S \subset \mathcal{X}$, and $x \in \mathcal{X} \setminus S$, it holds that

$$\Pr[\text{PRF.Eval}(K, x) = \text{PRF.Eval}(K\{S\}, x) : K \leftarrow \text{PRF.Gen}(1^\lambda), \\ K\{S\} \leftarrow \text{PRF.Punct}(K, S)] = 1.$$

- **Pseudorandom at punctured points:** For all two-phase PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ playing the Figure 1 game, it holds that

$$|\Pr[\text{PPRF-SEC}(1^\lambda, 0) = 1] - \Pr[\text{PPRF-SEC}(1^\lambda, 1) = 1]| \leq \text{negl}(\lambda).$$

Game PPRF-SEC($1^\lambda, b$): $(S, st) \leftarrow \mathcal{A}_1(1^\lambda)$ $K \leftarrow \text{PRF.Gen}(1^\lambda); K\{S\} \leftarrow \text{PRF.Punct}(K, S)$ $\mathbf{y}_0 := \text{PRF.Eval}(K, S); \mathbf{y}_1 \leftarrow \mathcal{Y}^{\times S }$ $b' \leftarrow \mathcal{A}_2(st, S, K\{S\}, \mathbf{y}_b)$ Return $\llbracket b = b' \rrbracket$

Fig. 1: Game to define PPRF security at punctured points.

A PPRF can be constructed from any length-doubling pseudorandom generator, using the GGM PRF construction [30].

Distributed PPRF Correlation Functionality. We consider a two-party functionality between a sender and a receiver described in Figure 2. This functionality is parameterized by a PPRF whose domain is $\{0, 1\}^\ell$, key space is

$\{0, 1\}^\lambda$, and the range is $\mathbb{F}_{p^r}$. The sender sends a value $\beta \in \mathbb{F}_{p^r}$ to the functionality and the receiver sends a punctured point $\alpha \in \{0, 1\}^\ell$. The functionality generates a PPRF key $K \leftarrow \text{PRF.Gen}(1^\lambda)$ and sends this to the sender. It then computes the punctured key $K\{\alpha\} \leftarrow \text{PRF.Punct}(K, \{\alpha\})$ and delivers this to the receiver along with $\beta - \text{PRF.Eval}(K, \alpha)$. Boyle *et al.* [12] showed that this functionality can be computed with semi-honest security from oblivious transfer (OT) [42] assuming black-box access to a length-doubling PRG. The formal theorem statement is given below.

Theorem 1 (Theorem 7 of [12]). *Assuming a black-box access to a length-doubling PRG $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$, there exists a 2-party protocol for $\mathcal{F}_{\text{PPRF}}$ with semi-honest security in the $\mathcal{F}_{\text{OT}}$ -hybrid model. The computational complexity is dominated by $O(2^\ell)$ calls to G . The interaction consists of ℓ parallel calls to $\mathcal{F}_{\text{OT}}$ and requires communication of $(2\lambda + 1)\ell + 2r \log p + 1$ bits.*

Functionality $\mathcal{F}_{\text{PPRF}}[1^\lambda, \ell, p, r]$	
PARAMETERS:	$\lambda, \ell, p, r \in \mathbb{N}$. Let PPRF be a puncturable PRF with domain $\{0, 1\}^\ell$, key space $\{0, 1\}^\lambda$, and range $\mathbb{F}_{p^r}$. $\ell = \ell(\lambda)$ is logarithmic in λ .
INPUTS:	The sender has input $\beta \in \mathbb{F}_{p^r}$ and the receiver has input $\alpha \in \{0, 1\}^\ell$.
FUNCTIONALITY:	– $K \leftarrow \text{PRF.Gen}(1^\lambda)$ is sent to the sender. – $K\{\alpha\} \leftarrow \text{PRF.Punct}(K, \alpha)$ and $\tau := \beta - \text{PRF.Eval}(K, \alpha)$ are sent to the receiver.

Fig. 2: Distributed PPRF correlation ideal functionality $\mathcal{F}_{\text{PPRF}}$.

3.7 Equality Testing

Our construction uses a secure computation protocol for equality testing between two parties where the result of the test is secret shared among the two parties. Figure 3 models this functionality where we let $[x]_b$ denote party P_b 's additive share of x .

Couteau [19] gives a semi-honest secure protocol with pre-processing to realize the $\mathcal{F}_{=\text{-test}}$ functionality from oblivious transfer (OT) [42].

Theorem 2 (Theorem 2 of [19]). *There is a semi-honest secure protocol with pre-processing for $\mathcal{F}_{=\text{-test}}$ in the $\mathcal{F}_{\text{OT}}$ -hybrid model. For $b_0, b_1 \in \{0, 1\}^\ell$, the amortized communication complexity of over many calls to $\mathcal{F}_{=\text{-test}}$ is $O(\frac{\lambda}{\log \lambda} \cdot \ell)$ in the pre-processing phase and $O(\ell)$ in the online phase.*

Functionality: $\mathcal{F}_{\text{--test}}$	
INPUTS: $b_0, b_1 \in \{0, 1\}^\ell$ from P_0 and P_1 , respectively.	
FUNCTIONALITY:	
– If $b_0 = b_1$: Output $[1]_0$ to P_0 and $[1]_1$ to P_1. – If $b_0 \neq b_1$: Output $[0]_0$ to P_0 and $[0]_1$ to P_1.	

Fig. 3: Ideal functionality for equality testing with secret shared output.

4 Programmable Oblivious Linear Evaluation over $\mathbb{F}_2$

In this section, we define the functionality for programmable oblivious linear evaluation (OLE) and give an efficient protocol for it. The protocol implicitly provides a PCG with distributed seed generation for programmable OLE as discussed in Section 4.2. Recall that this implies a PCG for multiparty Beaver triples [13] (Appendix A).

4.1 Definition of the Functionality

We give the formal description of the programmable OLE functionality in Figure 4. This functionality is parameterized by a specific function PRG, which we describe below.

Description of PRG. Let $n = n(\lambda)$ and $k = k(\lambda)$. Following the notation in Definitions 1 and 2, let $\mathbf{A} \in \mathbb{Z}_2^{N \times n}$ be the output of a code generator $C(\mathbb{F}_2, n, N)$ and let $\mathbf{H} \in \mathbb{Z}_2^{n \times k}$ be the output of the dual code generator $D(\mathbb{F}_2, k, n - k)$. Let $\chi(\mathbb{F}_2, n, N)$ and $\chi(\mathbb{F}_2, k, n - k)$ be regular error distributions with Hamming weights $t_1 = t_1(\lambda)$ and $t_2 = t_2(\lambda)$ respectively. In other words, errors are sampled from $\text{Reg}_{t_1}^N(\mathbb{F}_2)$ and $\text{Reg}_{t_2}^k(\mathbb{F}_2)$. The input to PRG consists of two vectors $(\hat{\mathbf{e}}, \mathbf{e})$, where $\hat{\mathbf{e}}$ is sampled from $\chi(\mathbb{F}_2, k, n - k)$ and $\mathbf{e}$ is sampled from $\chi(\mathbb{F}_2, n, N)$. On input a seed $r = (\hat{\mathbf{e}}, \mathbf{e})$, the function computes $\text{PRG}(r) := \mathbf{A} \cdot (\mathbf{H} \cdot \hat{\mathbf{e}}) \oplus \mathbf{e}$. Although the length of the vector $\mathbf{e}$ is N , it has a succinct description (of length $s(\lambda)$).

Not that for the purpose of the ideal functionality and protocol in this section, we only require PRG to be a public deterministic function. However, when the generated OLE correlations are used in an MPC protocol, we will rely on the function PRG to be a pseudorandom generator, and the pseudorandomness of PRG relies on the primal and dual LPN assumptions.

Since a full simulation-based definition is impossible to achieve [12], we follow the definition from [12] where the corrupted parties can influence their outputs in the ideal functionality.

4.2 Construction

The construction is subdivided into several components. Note that

$$\mathbf{a} \odot \mathbf{b} = \text{PRG}(r_0) \odot \text{PRG}(r_1)$$

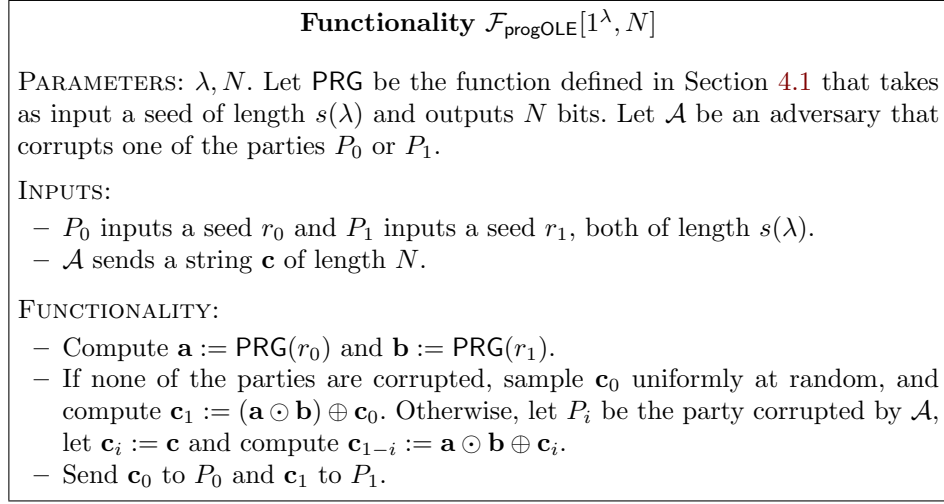


Fig. 4: Ideal functionality of two-party programmable OLE.

$$\begin{aligned}
&= \left(\mathbf{A} \cdot \underbrace{(\mathbf{H} \cdot \hat{\mathbf{e}}_0)}_{\doteq \mathbf{s}_0} \oplus \mathbf{e}_0 \right) \odot (\mathbf{A} \cdot (\mathbf{H} \cdot \hat{\mathbf{e}}_1) \oplus \mathbf{e}_1) \\
&= (\mathbf{A} \cdot \mathbf{s}_0 \oplus \mathbf{e}_0) \odot \left(\underbrace{(\mathbf{A} \cdot \mathbf{H}) \cdot \hat{\mathbf{e}}_1}_{\doteq \mathbf{B}} \oplus \mathbf{e}_1 \right) \\
&= (\mathbf{A} \cdot \mathbf{s}_0 \oplus \mathbf{e}_0) \odot (\mathbf{B} \cdot \hat{\mathbf{e}}_1 \oplus \mathbf{e}_1) \\
&= (\mathbf{A} \cdot \mathbf{s}_0) \odot (\mathbf{B} \cdot \hat{\mathbf{e}}_1) \oplus \mathbf{e}_0 \odot (\mathbf{B} \cdot \hat{\mathbf{e}}_1) \oplus \mathbf{e}_1 \odot (\mathbf{A} \cdot \mathbf{s}_0) \oplus \mathbf{e}_0 \odot \mathbf{e}_1 \\
&= (\mathbf{A} \cdot \mathbf{s}_0) \odot (\mathbf{B} \cdot \hat{\mathbf{e}}_1) \oplus \mathbf{e}_0 \odot (\mathbf{A} \cdot \underbrace{(\mathbf{H} \cdot \hat{\mathbf{e}}_1)}_{\mathbf{s}_1}) \oplus \mathbf{e}_1 \odot (\mathbf{A} \cdot \mathbf{s}_0) \oplus \mathbf{e}_0 \odot \mathbf{e}_1 \\
&= \underbrace{(\mathbf{A} \cdot \mathbf{s}_0) \odot (\mathbf{B} \cdot \hat{\mathbf{e}}_1)}_{\boxed{1}} \oplus \underbrace{\mathbf{e}_0 \odot (\mathbf{A} \cdot \mathbf{s}_1)}_{\boxed{2}} \oplus \underbrace{\mathbf{e}_1 \odot (\mathbf{A} \cdot \mathbf{s}_0)}_{\boxed{3}} \oplus \underbrace{\mathbf{e}_0 \odot \mathbf{e}_1}_{\boxed{4}}. \tag{3}
\end{aligned}$$

We will give a protocol for computing additive shares of $\boxed{1}$, $\boxed{2}$, $\boxed{3}$, and $\boxed{4}$ respectively. This will directly give a protocol for computing additive secret shares of $\mathbf{a} \odot \mathbf{b}$.

- **Computing additive shares of $\boxed{1}$:** For each $i \in [N]$, the i -th element of $\boxed{1}$ can be computed as $\langle \mathbf{a}_i \otimes \mathbf{b}_i, \mathbf{s}_0 \otimes \hat{\mathbf{e}}_1 \rangle$, where $\mathbf{a}_i$ and $\mathbf{b}_i$ denote the i -th row of $\mathbf{A}$ and $\mathbf{B}$, respectively. Since matrices $\mathbf{A}, \mathbf{B}$ are known to both parties, it suffices to compute additive shares of $\mathbf{s}_0 \otimes \hat{\mathbf{e}}_1$.

Let $\hat{\mathbf{e}}_1 = (\hat{e}_{1,1}, \dots, \hat{e}_{1,k})$. We view $\mathbf{s}_0 \otimes \hat{\mathbf{e}}_1$ as a $k \times n$ matrix $\mathbf{M}$ given by:

$$\mathbf{M} := \mathbf{s}_0 \otimes \hat{\mathbf{e}}_1 = \begin{pmatrix} \hat{e}_{1,1} \cdot \mathbf{s}_0^\top \\ \hat{e}_{1,2} \cdot \mathbf{s}_0^\top \\ \vdots \\ \hat{e}_{1,k} \cdot \mathbf{s}_0^\top \end{pmatrix}.$$

Since $\hat{\mathbf{e}}_1$ comes from the regular error distribution with Hamming weight t_2 , we can view $\hat{\mathbf{e}}_1$ as a concatenation of t_2 blocks where each block is a random unit vector of size k/t_2 . Let $\hat{\mathbf{e}}_1$ be described by $(i_1, \dots, i_{t_2})$ where each $i_j \in [k/t_2]$ and i_j denotes the index in the j -th block where there is a one.

1. For each $j \in [t_2]$,
 - (a) The parties call the $\mathcal{F}_{\text{PPRF}}[1^\lambda, \log(k/t_2), 2, n]$ with P_0 (as sender) providing $\beta = \mathbf{s}_0$ as input and P_1 (as receiver) providing $\alpha = i_j$ as input. P_0 receives a PPRF key K_j as output and P_1 receives $K_j\{i_j\}$ and τ_j as output.
 - (b) The parties perform a full domain evaluation of the PPRF where P_1 sets the evaluation at i_j as τ_j .
2. P_0 and P_1 concatenate all the matrices obtained above to compute matrices $[\mathbf{M}]_0$ and $[\mathbf{M}]_1$ such that $[\mathbf{M}]_0 \oplus [\mathbf{M}]_1 = \mathbf{s}_0 \otimes \hat{\mathbf{e}}_1$.
3. For all $b \in \{0, 1\}$ and $i \in [N]$, P_b computes $\langle \mathbf{a}_i \otimes \mathbf{b}_i, [\mathbf{M}]_b \rangle$. Here, both $\mathbf{a}_i \otimes \mathbf{b}_i$ and $[\mathbf{M}]_b$ are viewed as vectors of length $n \cdot k$.
4. P_0 and P_1 concatenate all the values computed in Step 3 to obtain length N vectors $\mathbf{x}_0^{(1)}, \mathbf{x}_1^{(1)}$ respectively. It follows that

$$\mathbf{x}_0^{(1)} \oplus \mathbf{x}_1^{(1)} = (\mathbf{A} \cdot \mathbf{s}_0) \odot (\mathbf{B} \cdot \hat{\mathbf{e}}_1).$$

- **Computing additive shares of [2].** Observe that as $\mathbf{e}_0$ comes from the regular error distribution with Hamming weight t_1 , we can view $\mathbf{e}_0$ as concatenation of t_1 blocks where each block is a random unit vector of size N/t_1 . Let $\mathbf{e}_0$ be described by $(i_1, \dots, i_{t_1})$ where each $i_j \in [N/t_1]$ and i_j denotes the index within the j -th block where there is a one.

1. P_1 samples $\mathbf{k} \leftarrow \text{AHE.Gen}(1^\lambda)$, computes $c \leftarrow \text{AHE.Enc}(\mathbf{k}, \mathbf{s}_1)$, and sends c to P_0 .
2. For each $j \in [t_1]$, P_0 samples a random bit α_j and computes $c'_j \leftarrow \text{AHE.Eval}(c, f_j)$ where f_j is the affine function that on input $\mathbf{s}$ outputs $\langle \mathbf{a}_{(j-1)N/t_1+i_j}, \mathbf{s} \rangle \oplus \alpha_j$. P_0 sends $c'_1, \dots, c'_{t_1}$ to P_1 .
3. For each $j \in [t_1]$,
 - (a) P_1 decrypts c'_j using $\mathbf{k}$ to obtain β_j .
 - (b) P_0 and P_1 invoke the $\mathcal{F}_{\text{PPRF}}[1^\lambda, \log(N/t_1), 2, 1]$ with P_1 (as sender) providing β_j as input and P_0 (as receiver) providing i_j as input. P_1 receives a PPRF key K_j as output and P_0 receives $K_j\{i_j\}$ and τ_j as output.
 - (c) The parties perform a full domain evaluation of the PPRF where P_0 sets the evaluation at i_j as $\tau_j \oplus \alpha_j$.

4. P_0 and P_1 concatenate all the vectors computed in Step 3 to obtain length N vectors $\mathbf{x}_0^{(2)}, \mathbf{x}_1^{(2)}$ respectively. It follows that

$$\mathbf{x}_0^{(2)} \oplus \mathbf{x}_1^{(2)} = \mathbf{e}_0 \odot (\mathbf{A} \cdot \mathbf{s}_1).$$

- **Computing additive shares of [3]:** This is symmetric to computing additive shares of [2]. Specifically, the two parties obtain length N vectors $\mathbf{x}_0^{(3)}, \mathbf{x}_1^{(3)}$ respectively such that

$$\mathbf{x}_0^{(3)} \oplus \mathbf{x}_1^{(3)} = \mathbf{e}_1 \odot (\mathbf{A} \cdot \mathbf{s}_0).$$

- **Computing additive shares of [4]:** Recall that $\mathbf{e}_0$ and $\mathbf{e}_1$ are sampled from regular error distributions with Hamming weight t_1 . Let $\{i_1, \dots, i_{t_1}\}$ and $\{i'_1, \dots, i'_{t_1}\}$ be the positions where there is a one in $\mathbf{e}_0$ and $\mathbf{e}_1$, respectively.

1. For each $j \in [t_1]$,
 - (a) P_0 and P_1 call $\mathcal{F}_{\text{--test}}$ with inputs i_j and i'_j respectively. The parties respectively receive b_j and b'_j from the ideal functionality.
 - (b) P_0 and P_1 invoke $\mathcal{F}_{\text{PPRF}}[1^\lambda, \log(N/t_1), 2, 1]$ with P_1 (as sender) providing b'_j as input and P_0 (as receiver) providing i_j as input. P_1 receives a PPRF key K_j and P_0 receives $K_j\{i_j\}$ and τ_j .
 - (c) The parties perform a full domain evaluation of the PPRF where P_0 sets the evaluation at i_j as $\tau_j \oplus b_j$.
2. P_0 and P_1 concatenate all the vectors computed in Step 1c to obtain length N vectors $\mathbf{x}_0^{(4)}, \mathbf{x}_1^{(4)}$ respectively. It follows that

$$\mathbf{x}_0^{(4)} \oplus \mathbf{x}_1^{(4)} = \mathbf{e}_0 \odot \mathbf{e}_1.$$

- **Computing output:** For each $b \in \{0, 1\}$, party P_b computes the output vector $\mathbf{c}_b = \bigoplus_{i \in [4]} \mathbf{x}_b^{(i)}$.

PCG Interpretation. This protocol implicitly gives a PCG with distributed seed generation for two-party programmable OLE. The PCG seeds for the two parties are the PPRF outputs. Specifically, P_0 's seed consists of t_2 PPRF keys K_j in [1] (Step 1a), t_1 PPRF punctured keys $K_j\{i_j\}$ and τ_j in [2] (Step 3b), t_1 PPRF keys K_j in [3] (Step 3b), and t_1 PPRF punctured keys $K_j\{i_j\}$ and τ_j in [4] (Step 1b). P_1 's seed consists of t_2 PPRF punctured keys $K_j\{i_j\}$ and τ_j in [1] (Step 1a), t_1 PPRF keys K_j in [2] (Step 3b), t_1 PPRF punctured keys $K_j\{i_j\}$ and τ_j in [3] (Step 3b), and t_1 PPRF keys K_j in [4] (Step 1b).

PCG for Multiparty Beaver Triples. PCG for two-party programmable OLE is known to imply PCG for multiparty Beaver triples [13]. For completeness, we present the construction in Appendix A.

4.3 Proof of Security

Theorem 3. *Assuming PPRF and AHE security (Def. 5 and 3), construction 4.2 is a semi-honest secure realization of $\mathcal{F}_{\text{progOLE}}$ (Figure 4) in the $(\mathcal{F}_{\text{PPRF}}, \mathcal{F}_{\text{=test}})$ -hybrid model.*

To prove Theorem 3, we consider three cases: when both parties are honest, when P_0 is corrupted, and when P_1 is corrupted.

P_0 and P_1 are honest. In this case, we show that the output shares $(\mathbf{c}_0, \mathbf{c}_1)$ are computationally indistinguishable from two random strings subject to their sum being equal to $\mathbf{a} \odot \mathbf{b}$.

- Hyb₀ : This corresponds to the real execution of the protocol.
- Hyb₁ : In this hybrid, we replace strings obtained by both parties as a result of the full domain evaluation on the outputs of every $\mathcal{F}_{\text{PPRF}}(\beta, \alpha)$ with uniformly random strings such that their sum is equal to full domain evaluation of the point function $F_{\alpha, \beta}$ (the point function $F_{\alpha, \beta}$ outputs β on input α and outputs 0 everywhere else). Hyb₁ is computationally indistinguishable from Hyb₀ by the pseudorandomness of the PRF.

We argue indistinguishability through a series of intermediate hybrids, where in each hybrid, we replace the output strings from one $\mathcal{F}_{\text{PPRF}}(\beta, \alpha)$ call. To be precise, we define $m := 3t_1 + t_2$ intermediate hybrids Hyb_{1,0}, Hyb_{1,1}, Hyb_{1,2}, ..., Hyb_{1,m}, where Hyb_{1,0} = Hyb₀ and Hyb_{1,m} = Hyb₁. The k -th intermediate hybrid replaces the output strings from the k -th call to $\mathcal{F}_{\text{PPRF}}(\beta, \alpha)$ with random secret sharing of the point function $F_{\alpha, \beta}$. Suppose by way of contradiction that there exists a PPT distinguisher $\mathcal{D}_1$ that can distinguish the output shares between Hyb_{1,k-1} and Hyb_{1,k} with non-negligible probability for some $k \in [m]$. The difference between Hyb_{1,k-1} and Hyb_{1,k} is specified in Fig. 5 (top). Note that $\mathcal{D}_1$ is an outside party that can only view public parameters and the final shares computed by P_0 and P_1 , but in Fig. 5 we allow $\mathcal{D}_1$ to have $\mathbf{x}_0$ and $\mathbf{x}_1$ as inputs, which is a stronger assumption. To show that the distributions of shares in both hybrids are computationally indistinguishable, consider the PRF adversary $\mathcal{A}_1$ in Figure 5 (bottom) simulating one of the two hybrids. If $\mathcal{A}_1$ has access to a PRF, then the generated outputs are the same as Hyb_{1,k-1}; if $\mathcal{A}_1$ has access to a random function, then the outputs are the same as Hyb_{1,k}. The outputs are then passed to the distinguisher $\mathcal{D}$ who guesses between Hyb_{1,k-1} and Hyb_{1,k}. Since $\mathcal{D}_1$ is correct with non-negligible probability, so is the PRF adversary $\mathcal{A}_1$.

- Hyb₂ : In this hybrid, we make a syntactic change. We sample random string $\mathbf{y}_0$ and set $\mathbf{y}_1$ to be $\mathbf{a} \odot \mathbf{b} \oplus \mathbf{y}_0$. This hybrid is identical to the previous one as the sum of additive shares of $\boxed{1}$, $\boxed{2}$, $\boxed{3}$, and $\boxed{4}$ are randomly distributed subject to their sum being equal to $\mathbf{a} \odot \mathbf{b}$.

P_0 is corrupted. We first construct a PPT simulator that simulates P_0 's view. The simulator runs the honest P_0 protocol to generate its view with the following exceptions: For every $\mathcal{F}_{\text{PPRF}}(\beta, \alpha)$ where P_0 is the sender, generate

Hyb_{1,k-1}($1^\lambda, \alpha, \beta$): $(K, (K\{\alpha\}, \tau)) \leftarrow \mathcal{F}_{\text{PPRF}}[1^\lambda, \ell, p, r](\beta, \alpha)$ For all $i \in \{0, 1\}^\ell, i \neq \alpha$: $x_{0,i} := \text{PRF.Eval}(K, i)$ $x_{1,i} := \text{PRF.Eval}(K\{\alpha\}, i)$ For $i = \alpha$: $x_{0,\alpha} := \text{PRF.Eval}(K, \alpha)$ $x_{1,\alpha} := \beta - x_{0,\alpha}$ Return $(\mathbf{x}_0, \mathbf{x}_1)$	Hyb_{1,k}($1^\lambda, \alpha, \beta$): $(K, (K\{\alpha\}, \tau)) \leftarrow \mathcal{F}_{\text{PPRF}}[1^\lambda, \ell, p, r](\beta, \alpha)$ For all $i \in \{0, 1\}^\ell, i \neq \alpha$: $x_{0,i} \leftarrow \mathbb{F}_{p^r}; x_{1,i} := x_{0,i}$ For $i = \alpha$: $x_{0,\alpha} \leftarrow \mathbb{F}_{p^r}; x_{1,\alpha} := \beta - x_{0,\alpha}$ Return $(\mathbf{x}_0, \mathbf{x}_1)$
---	--

Adversary $\mathcal{A}_1^{F(\cdot)}(1^\lambda, \alpha, \beta)$

For all $i \in \{0, 1\}^\ell, i \neq \alpha$:

$x_{0,i} := F(i); c_{1,i} := x_{0,i}$

For $i = \alpha$:

$x_{0,\alpha} := F(\alpha); x_{1,\alpha} := \beta - y_b$

$b' \leftarrow \mathcal{D}_1(1^\lambda, \mathbf{x}_0, \mathbf{x}_1)$

Return b' .

Fig. 5: Top left: use of $\mathcal{F}_{\text{PPRF}}$ and full domain evaluation in **Hyb_{1,k-1}**. Top right: altered full domain evaluation in **Hyb_{1,k}**. Bottom: PRF adversary simulating **Hyb_{1,k-1}** or **Hyb_{1,k}** for distinguisher $\mathcal{D}_1$.

$K \leftarrow \text{PRF.Gen}(1^\lambda)$ and output K to P_0 . For every $\mathcal{F}_{\text{PPRF}}(\beta, \alpha)$ where P_0 is the receiver, generate $K \leftarrow \text{PRF.Gen}(1^\lambda)$, $K\{\alpha\} \leftarrow \text{PRF.Punct}(K, \alpha)$, sample a uniform random value τ , and output $(K\{\alpha\}, \tau)$ to P_0 . When computing [2], replace the AHE encryption of $\mathbf{s}_1$ sent from P_1 with a ciphertext encrypting all zeroes. When computing [3], replace the AHE evaluations sent from P_1 with fresh AHE encryptions of random bits. When computing [4], set the output from each $\mathcal{F}_{\text{=test}}$ to be a random bit. Finally, compute $\mathbf{c}_0$ as specified in the protocol and send it to the ideal functionality.

Next, we argue that P_0 's view together with P_1 's output in the real execution is computationally indistinguishable from P_0 's simulated view together with P_1 's output in the ideal world via a series of hybrids.

- **Hyb₀** : This corresponds to the real execution of the protocol.
- **Hyb₁** : In this hybrid, we make a syntactic change. Instead of computing P_1 's shares in the computation of terms [1], [2], [3], and [4], we add all P_0 's shares in these computations to obtain $\mathbf{c}_0$ and set P_1 's final share to be $\mathbf{c}_1 := \mathbf{a} \odot \mathbf{b} \oplus \mathbf{c}_0$. Note that since we are in the semi-honest security model, this change is only syntactic and hence, this hybrid is identically distributed to **Hyb₀**.
- **Hyb₂** : In this hybrid, for every $\mathcal{F}_{\text{PPRF}}(\beta, \alpha)$ where P_0 is the receiver, we replace the output τ with a uniformly chosen random value. This hybrid is

computationally indistinguishable from Hyb_1 following the pseudorandomness at punctured points of PPRF.

We argue indistinguishability through a series of intermediate hybrids, where in each hybrid, we replace the output from one $\mathcal{F}_{\text{PPRF}}(\beta, \alpha)$ call. To be precise, we define $m := 2t_1$ intermediate hybrids $\text{Hyb}_{2,0}, \text{Hyb}_{2,1}, \dots, \text{Hyb}_{2,m}$, where $\text{Hyb}_{2,0} = \text{Hyb}_1$ and $\text{Hyb}_{2,m} = \text{Hyb}_2$. The k -th intermediate hybrid replaces the output τ from the k -th call to $\mathcal{F}_{\text{PPRF}}(\beta, \alpha)$ in steps [2] and [4] with a uniform random value. Suppose by way of contradiction that there exists a PPT distinguisher $\mathcal{D}_2$ that can distinguish between $\text{Hyb}_{2,k-1}$ and $\text{Hyb}_{2,k}$ with non-negligible probability for some $k \in [m]$.

We construct a PPT adversary $\mathcal{A}_2$ to break the PPRF security game in Fig. 1. On input $(st, \alpha, K\{\alpha\}, y_b)$ in the security game, $\mathcal{A}_2$ follows the protocol in the same way as $\text{Hyb}_{2,k-1}$ except that in the k -th call to $\mathcal{F}_{\text{PPRF}}(\beta, \alpha)$, replace the output to P_0 by $(K\{\alpha\}, \beta - y_b)$. Finally, $\mathcal{A}_2$ passes the generated output to $\mathcal{D}_2$ and outputs whatever $\mathcal{D}_2$ outputs. If $y_b = \text{PRF.Eval}(K, \alpha)$, then the generated output is the same as $\text{Hyb}_{1,k-1}$; if y_b is randomly sampled, then the output is the same as $\text{Hyb}_{1,k}$. Since $\mathcal{D}_2$ is correct with non-negligible probability, so is $\mathcal{A}_2$.

- Hyb₃ : In this hybrid, we replace the AHE encryptions of $\mathbf{s}_1$ sent from P_1 in computing additive shares of [2] with a ciphertext encrypting all zeroes. This hybrid is computationally indistinguishable from the previous following the IND-CPA security of AHE.

Suppose by way of contradiction that there exists a PPT distinguisher $\mathcal{D}_3$ that can distinguish between Hyb_2 and Hyb_3 with non-negligible probability. We construct a PPT adversary $\mathcal{A}_3$ to break the IND-CPA security of AHE. $\mathcal{A}_3$ follows the protocol in the same way as Hyb_2 except that the AHE encryptions of $\mathbf{s}_1$ is replaced with a challenge ciphertext $c \leftarrow \text{AHE.Enc}(k, \mathbf{s})$, where $\mathbf{s}$ is either $\mathbf{s}_1$ or $\mathbf{0}$. $\mathcal{A}_3$ passes the generated output to $\mathcal{D}_3$ and outputs whatever $\mathcal{D}_3$ outputs. If c is an encryption of $\mathbf{s}_1$, then the generated output is Hyb_2 ; otherwise the generated output is Hyb_3 . Since $\mathcal{D}_3$ is correct with non-negligible probability, so is $\mathcal{A}_3$.

- Hyb₄ : In this hybrid, we replace the AHE evaluations of $\mathbf{s}_1$ sent from P_1 in computing additive shares of [3] with fresh AHE encryptions of random bits. This hybrid is indistinguishable from Hyb_3 following the function privacy of AHE. Since the AHE evaluation computes inner product plus a random bit, the resulting ciphertext is an encryption of a random bit. By function privacy of AHE, this is statistically indistinguishable from a fresh encryption of a random bit.
- Hyb₅ : In this hybrid, set the output from each $\mathcal{F}_{\text{PPRF}}$ to be a random bit. This hybrid has identical distribution with Hyb_4 . This gives the simulated view and P_1 's output in the ideal world.

P_1 is corrupted. We construct a PPT simulator that simulates P_1 's view. The simulator runs the honest P_1 protocol to generate its view with the following exceptions: For every $\mathcal{F}_{\text{PPRF}}(\beta, \alpha)$ where P_1 is the sender, generate $K \leftarrow$

$\text{PRF.Gen}(1^\lambda)$ and output K to P_1 . For every $\mathcal{F}_{\text{PPRF}}(\beta, \alpha)$ where P_1 is the receiver, generate $K \leftarrow \text{PRF.Gen}(1^\lambda)$, $K\{\alpha\} \leftarrow \text{PRF.Punct}(K, \alpha)$, sample a uniform random value τ , and output $(K\{\alpha\}, \tau)$ to P_1 . When computing [2], replace the AHE evaluations sent from P_0 with fresh AHE encryptions of random bits. When computing [3], replace the AHE encryption of s_0 sent from P_0 with a ciphertext encrypting all zeroes. When computing [4], set the output from each $\mathcal{F}_{\text{--test}}$ to be a random bit. Finally, compute c_1 as specified in the protocol and send it to the ideal functionality. The indistinguishability proof is similar to the proof of when P_0 is corrupted.

5 Implementation and Evaluation

We implement our protocol for two-party programmable OLE in C++ and report the performance in this section. Our code is available as supplementary materials and will be open-sourced.

5.1 Implementation Details and Optimizations

In this section, we discuss the instantiations of the building blocks in our protocol and optimizations to further improve the concrete efficiency. Our protocol requires the following building blocks:

- A distributed PPRF protocol realizing $\mathcal{F}_{\text{PPRF}}[1^\lambda, \ell, p, r]$
- An equality testing protocol $\mathcal{F}_{\text{--test}}$
- A function-private additively homomorphic private-key encryption AHE

We instantiate distributed PPRF with the protocol from [12] and equality testing with the protocol from [19], both based on OT.

Combined PPRF. To compute additive shares of [2] and [4], parties run t_1 instances of $\mathcal{F}_{\text{PPRF}}[1^\lambda, \log(N/t_1), 2, 1]$ (Step 3b in [2] and Step 1b in [4]) where P_1 is the sender and P_0 is the receiver. Note that in both cases, P_0 provides the same input i_j . We take advantage of this to combine these two $\mathcal{F}_{\text{PPRF}}$ calls into one where P_0 's input remains i_j and P_1 's input becomes $\beta_j \oplus b'_j$. In this way, we combine [2] and [4] into a single PPRF that provides shares of $[2] \oplus [4]$. This optimization halves the PPRF-related computation and communication costs for those terms.

Additively Homomorphic Encryption. To compute additive shares of [2] and [3], our protocol uses an additively homomorphic private-key encryption scheme AHE with function privacy for addition over the binary field. We instantiate AHE with exponential ElGamal encryption [26] with discrete Gaussian noise, given by Brakerski *et al.* [15]. We adopt this scheme for its low communication cost. Recall that in (public-key) exponential ElGamal encryption, $\text{Enc}(m) = (g^r, h^r \cdot g^m)$ where g is a group generator and $h = g^s$ is part of the

public key (with a secret key s). One ciphertext contains two group elements, or 528 bits in our implementation.

This can be further improved upon as we only require a private-key encryption scheme. Hence the ElGamal ciphertext can be computed as $\text{Enc}(m) = (g', (g')^s \cdot g^m)$ for a randomly sampled group element g' . Both parties can sample g' from a random oracle, so the encryptor only needs to send the second component of the ciphertext, reducing the communication cost in Step 1 of [2] and [3] by a factor of two.

Computing Additive Shares of [1]. When computing the additive shares of [1], in Step 3, for each triple $i \in [N]$, each party P_b computes $\langle \mathbf{a}_i \otimes \mathbf{b}_i, [\mathbf{M}]_b \rangle$. Naïvely computing the inner product requires $n \cdot k$ AND and XOR operations over bits. For the parameters we choose (Section 5.4), this requires at least 2^{40} bit operations *per triple* for generating $N = 2^{30}$ triples, becoming a bottleneck in the computation.

However, notice that we can leverage sparse LPN [2] to generate a sparse vector $\mathbf{a}_i$. Let z be the Hamming weight of $\mathbf{a}_i$, then the number of relevant (potentially non-zero) bits in $\mathbf{a}_i \otimes \mathbf{b}_i$ is only $z \cdot k$. We only need to consider these relevant bits in $[\mathbf{M}]_b$, which are the z columns specified by non-zero entries of $\mathbf{a}_i$. To compute the inner product $\langle \mathbf{a}_i \otimes \mathbf{b}_i, [\mathbf{M}]_b \rangle$, it suffices to take out these z columns of $[\mathbf{M}]_b$, XOR them, and then take an inner product with $\mathbf{b}_i$. This reduces the computation to $z \cdot k$ XOR and k AND operations over bits. If we store $\mathbf{b}_i$ and $[\mathbf{M}]_b$ in memory in the “column format” where each column is stored in contiguous memory, the computation cost will be $\frac{z \cdot k}{w}$ XOR and $\frac{k}{w}$ AND operations over words where w is the CPU word length. This optimization requires a one-time matrix transpose for $[\mathbf{M}]_b$. Concretely, for generating $N = 2^{30}$ triples, this step requires 196.6K XOR and 32.8K AND operations per triple.

5.2 Parameters

We have the following parameters in our protocol.

- λ : computational security parameter, set to 128.
- σ : statistical security parameter, set to 40.
- N : number of OLE correlations.
- n : length of $\mathbf{s}$, the secret vector for primal LPN. Note that the dimension of the primal LPN matrix $\mathbf{A}$ is $N \times n$.
- t_1 : Hamming weight of $\mathbf{e}$, the primal LPN error vector.
- k : length of the error vector $\hat{\mathbf{e}}$ for dual LPN. Note that the dimension of the dual LPN matrix $\mathbf{H}$ is $n \times k$.
- t_2 : Hamming weight of the error vector $\hat{\mathbf{e}}$ for dual LPN.
- z : Hamming weight of each row of $\mathbf{A}$.
- w : CPU word size, set to 64.

The concrete parameters are presented in Section 5.4. We obtain the primal and dual LPN parameters following the script provided by Liu *et al.* [37]. The attacks considered in the script include pooled Gauss [27], statistical decoding [1, 18, 28, 39], information set decoding [5, 25, 38, 41, 47], and the recent algebraic attack on LPN with regular noise [16]. The parameter z of sparse LPN is calculated based on [2]. We assume sparse LPN has the same level of security as standard LPN for an appropriately sampled $\mathbf{A}$ (see Remark 4.1 in [22] on how to sample $\mathbf{A}$). Additionally, we derive the standard deviation for the discrete Gaussian noise used in AHE based on [29].

5.3 Cost Analysis

In this section, we first analyze the communication cost of our distributed seed generation protocol. Then we consider the PCG interpretation of our protocol and analyze the PCG seed size and computational cost of PCG expansion.

Communication Cost. The communication for term [1] consists of t_2 calls to $\mathcal{F}_{\text{PPRF}}[1^\lambda, \log(k/t_2), 2, n]$, which by Theorem 1, has cost $t_2 \cdot (2\lambda \cdot \log(k/t_2) + 2n)$. Terms [2] and [3] both involve exchanging $(n + t_1)$ AHE ciphertexts and t_1 calls to $\mathcal{F}_{\text{PPRF}}[1^\lambda, \log(N/t_1), 2, 1]$. As discussed earlier, each AHE ciphertext in Step 1 only requires one group element while each ciphertext in Step 2 requires two group elements. Hence the communication cost for [2] and [3] is $2 \cdot ((n + 2t_1) \cdot |\mathbb{G}| + t_1 \cdot 2\lambda \cdot \log(N/t_1))$, where $|\mathbb{G}|$ is the number of bits representing a single group element. Finally, term [4] involves t_1 calls to both $\mathcal{F}_{\text{--test}}$ and $\mathcal{F}_{\text{PPRF}}[1^\lambda, \log(N/t_1), 2, 1]$. As discussed earlier, the $\mathcal{F}_{\text{PPRF}}$ calls can be combined with the ones in [2], hence the remaining communication in [4] is $O\left(t_1 \cdot \frac{\lambda}{\log \lambda} \log(N/t_1)\right)$ according to Theorem 2.

PCG Seed Size. Recall that our protocol implicitly gives a two-party PCG for programmable OLE. The PCG seeds for the two parties are the PPRF keys and punctured keys, which we instantiate with GGM-PRF.

P_0 's seed consists of t_2 PPRF keys K_j for $\mathcal{F}_{\text{PPRF}}[1^\lambda, \log(k/t_2), 2, n]$ in [1], t_1 PPRF punctured keys $K_j\{i_j\}$ and τ_j for $\mathcal{F}_{\text{PPRF}}[1^\lambda, \log(N/t_1), 2, 1]$ in [2], and t_1 PPRF keys K_j for $\mathcal{F}_{\text{PPRF}}[1^\lambda, \log(N/t_1), 2, 1]$ in [3]. The total seed size for P_0 is $(t_1 + t_2) \cdot \lambda + t_1 \cdot (\log(N/t_1) \cdot \lambda + 1)$.

P_1 's seed consists of t_2 PPRF punctured keys $K_j\{i_j\}$ and τ_j for $\mathcal{F}_{\text{PPRF}}[1^\lambda, \log(k/t_2), 2, n]$ in [1], t_1 PPRF keys K_j for $\mathcal{F}_{\text{PPRF}}[1^\lambda, \log(N/t_1), 2, 1]$ in [2], and t_1 PPRF punctured keys $K_j\{i_j\}$ and τ_j for $\mathcal{F}_{\text{PPRF}}[1^\lambda, \log(N/t_1), 2, 1]$ in [3]. The total seed size for P_1 is $t_2 \cdot (\log(k/t_2) \cdot \lambda + n) + t_1 \cdot (\lambda + \log(N/t_1) \cdot \lambda + 1)$.

PCG Expansion Cost. The PCG expansion consists of two main components: (a) full domain evaluation of PPRFs, and (b) inner product $\langle \mathbf{a}_i \otimes \mathbf{b}_i, [\mathbf{M}]_b \rangle$. Part (a) involves full domain evaluation of t_2 PPRFs with domain size k/t_2 and range $\{0, 1\}^n$ and $2 \cdot t_1$ PPRFs with domain size N/t_1 and range $\{0, 1\}$. Hence the total number of PRG calls is $t_2 \cdot (k/t_2) \cdot (n/\lambda) + 2t_1 \cdot N/t_1$. The amortized PRG calls

per triple is $\frac{k \cdot n}{\lambda \cdot N} + 2$. As discussed in Section 5.1, the computation cost for part (b) per triple is $\frac{z \cdot k}{w}$ XOR and $\frac{k}{w}$ AND operations over words.

5.4 Performance

We use the `cryptoTools` library [45] for our underlying cryptographic primitives and `relic` [3] for elliptic curve operations in ElGamal encryption. Both the PPRF and equality testing protocols are constructed from OT, which we instantiate with the `libOTe` library [40] for generating random OT correlations with silent OT extension [13, 43]. Benchmarks are run on two Amazon AWS `m7a.32xlarge` virtual machines with 128 vCPU with maximum frequency 3.7 GHz and 512GB of RAM, which communicate over a LAN network. In our performance reported in this section, we assume **A** and **H** are pre-sampled and provided to both parties as public information.

Distributed Seed Generation. The performance of our distributed seed generation protocol for various parameter settings is presented in Table 1. We report the total communication cost (in MB) and per-triple communication cost (in bits). For the protocol running time, we report the end-to-end running time of the seed generation protocol (in seconds) and the time it takes for both parties to generate all the triples after the protocol, namely the full domain evaluation of the PPRFs (in hours).

As shown in the table, for generating the same number of triples, different LPN parameters achieve the same level of security, but there are tradeoffs between communication and computational costs. For instance, when $N = 2^{28}$, the first setting of parameters achieves 119.7 MB of communication while the third setting achieves 68.0 MB. This comes at the cost of increased computation. The seed generation protocol and expansion phase take 84.6s and 1.07 hours respectively for the first setting, while they take 169.0s and 2.39 hours respectively for the third setting.

Additionally, the per-triple communication decreases as the number of triples N grows. When $N = 2^{30}$, the first setting in the table achieves communication of 1.99 bits/triple. For another set of parameters, our estimated communication cost is 1.07 bits/triple. We expect this to drop below 1 bit/triple, namely sublinear in N , for larger values of N . For instance, when $N = 2^{32}$, one set of parameters achieves as low as 0.78 bits/triple based on our estimation.

PCG Interpretation. We report the PCG seed size and expansion cost in Table 2. `seed0`, `seed1` denote the PCG seeds of P_0, P_1 respectively (in MB). This is in comparison with N bits in MB. For PCG expansion, we estimate the amortized number of PRG calls and the number of XOR and AND operations per triple (K stands for thousand). This is in contrast with more than 2 million PRG calls to generate a single triple as estimated in [11]. Based on our experiments, we report the amortized PCG expansion time per triple in microseconds. Here again, for generating the same number of triples, different LPN parameters achieve various tradeoffs between seed size and expansion time.

N	n	t_1	k	t_2	z	Communication		Runtime	
						Total (MB)	Per Triple (b)	Protocol (s)	Expand (hr)
2^{28}	2^{18}	2^{17}	2^{20}	47	5	119.7	3.74	84.6	1.07
2^{28}	2^{18}	2^{17}	2^{21}	31	5	118.7	3.71	124.3	2.13
2^{28}	2^{19}	2^{15}	2^{21}	45	6	68.0	2.13	169.0	2.39
2^{29}	2^{19}	2^{16}	2^{21}	45	6	97.1	1.52	173.8	4.78
2^{29}	2^{19}	2^{16}	2^{22}	30	6	93.8*	1.47*	—	—
2^{30}	2^{19}	2^{18}	2^{21}	45	6	255.2	1.99	210.7	9.61
2^{30}	2^{20}	2^{16}	2^{22}	44	6	137.2*	1.07*	—	—
2^{32}	2^{21}	2^{18}	2^{23}	42	6	397.8*	0.78*	—	—
2^{34}	2^{22}	2^{19}	2^{24}	41	6	827.8*	0.40*	—	—
2^{35}	2^{23}	2^{19}	2^{25}	39	6	1154*	0.28*	—	—
2^{36}	2^{23}	2^{20}	2^{25}	39	6	1718*	0.21*	—	—
2^{37}	2^{24}	2^{20}	2^{26}	37	6	7340*	0.14*	—	—

Table 1: Communication cost and running time for our distributed seed generation protocol for two-party programmable OLE. Numbers with “*” are based on estimation rather than experiments. Cells with “—” denote settings where the programs run out of memory.

N	n	t_1	k	t_2	z	N bits in MB	Seed Size (MB)		Expansion Cost (Per Triple)		
							seed ₀	seed ₁	PRG calls	XOR+AND	Runtime (μ s)
2^{28}	2^{18}	2^{17}	2^{20}	47	5	32	24.0	25.5	10	98.3K	14.3
2^{28}	2^{18}	2^{17}	2^{21}	31	5	32	24.0	25.0	18	196.6K	28.5
2^{28}	2^{19}	2^{15}	2^{21}	45	6	32	7.0	9.8	34	229.4K	32.0
2^{29}	2^{19}	2^{16}	2^{21}	45	6	64	14.0	16.8	18	229.4K	32.1
2^{29}	2^{19}	2^{16}	2^{22}	30	6	64	14.0	15.9	34	458.8K	—
2^{30}	2^{19}	2^{18}	2^{21}	45	6	128	52.0	54.9	10	229.4K	32.2
2^{30}	2^{20}	2^{16}	2^{22}	44	6	128	15.0	20.5	34	458.8K	—
2^{32}	2^{21}	2^{18}	2^{23}	42	6	512	60.0	70.5	34	917.5K	—

Table 2: Seed size and expansion cost of our PCG for two-party programmable OLE. Cells with “—” denote settings where the programs run out of memory.

Comparison with FOLEAGE [7]. The recent work of Bombar *et al.* [7] gives a PCG for multiparty Beaver triples over $\mathbb{F}_4$, which can be turned into Beaver triples over $\mathbb{F}_2$ with a broadcast of one bit per party for each triple, hence the overall communication cost is *at least* n bits per triple, where n is the number of parties.⁷ Furthermore, we *ignore* the concrete cost of their distributed seed

⁷ We note that broadcast channels might not be available in practice. If we achieve a broadcast channel via point-to-point channels, a straightforward method is for each party to send its bit to all other parties, which requires a total communication of $n(n-1)$ bits per triple. However, their protocol also allows each party to send its bit to a single party, who combines these bits into a single bit and sends it back to all

generation phase, as they only provide an estimate for the number of OT calls required in this phase rather than a full implementation.

The total communication cost of our multiparty Beaver triples is $n(n-1)$ instances of two-party programmable OLE seed generation. For a small number of parties and large number of triples, we expect our overall communication cost to be lower. Specifically, based on our estimation in Table 1, for $N = 2^{34}$ triples and $n = 3$ parties, we require 2.4 bits of total communication per triple, while FOLEAGE requires 3 bits per triple, $1.24\times$ more expensive. For $N = 2^{37}$ triples and $n = 5$ parties, we require 2.8 bits of total communication per triple, while FOLEAGE requires 5 bits per triple, $1.79\times$ more expensive.

In terms of computational cost and throughput, FOLEAGE outperforms our construction. However, we would like to stress that they do not give a PCG for generating multiparty Beaver triples over $\mathbb{F}_2$, whereas our work provides the first practical construction. After seed generation, all operations in the expansion phase are *silent*.

Acknowledgments

Peihan Miao, Alice Murphy, and Max Tromanhauser were supported⁸ in part by NSF SaTC Award 2247352, NSF CAREER Award 2442384, Meta Research Award, Google Research Scholar Award, and Amazon Research Award. Akshayaram Srinivasan was supported in part by a NSERC Discovery Grant RGPIN-2024-03928.

References

1. Al Jabri, A.: A statistical decoding algorithm for general linear block codes. In: Honary, B. (ed.) 8th IMA International Conference on Cryptography and Coding. Lecture Notes in Computer Science, vol. 2260, pp. 1–8. Springer, Berlin, Heidelberg, Germany, Cirencester, UK (Dec 17–19, 2001). https://doi.org/10.1007/3-540-45325-3_1
2. Applebaum, B., Barak, B., Wigderson, A.: Public-key cryptography from different assumptions. In: Schulman, L.J. (ed.) 42nd Annual ACM Symposium on Theory of Computing. pp. 171–180. ACM Press, Cambridge, MA, USA (Jun 5–8, 2010). <https://doi.org/10.1145/1806689.1806714>
3. Aranha, D.F., Gouvêa, C.P.L., Markmann, T., Wahby, R.S., Liao, K.: RELIC is an Efficient Library for Cryptography. <https://github.com/relic-toolkit/relic>
4. Beaver, D.: Efficient multiparty protocols using circuit randomization. In: Feigenbaum, J. (ed.) Advances in Cryptology – CRYPTO’91. Lecture Notes in Computer Science, vol. 576, pp. 420–432. Springer, Berlin, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 11–15, 1992). https://doi.org/10.1007/3-540-46766-1_34

parties. This requires a total communication of $2(n-1)$ bits per triple. If we estimate the communication cost this way, our advantage would have been even greater.

⁸ Part of the work was completed by Alice Murphy during her visit to Brown University and by Max Tromanhauser during his time as a student.

5. Becker, A., Joux, A., May, A., Meurer, A.: Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding. In: Pointcheval, D., Johansson, T. (eds.) *Advances in Cryptology – EUROCRYPT 2012*. Lecture Notes in Computer Science, vol. 7237, pp. 520–536. Springer, Berlin, Heidelberg, Germany, Cambridge, UK (Apr 15–19, 2012). https://doi.org/10.1007/978-3-642-29011-4_31
6. Blum, A., Furst, M.L., Kearns, M.J., Lipton, R.J.: Cryptographic primitives based on hard learning problems. In: Stinson, D.R. (ed.) *Advances in Cryptology – CRYPTO’93*. Lecture Notes in Computer Science, vol. 773, pp. 278–291. Springer, Berlin, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 22–26, 1994). https://doi.org/10.1007/3-540-48329-2_24
7. Bombar, M., Bui, D., Couteau, G., Couvreur, A., Ducros, C., Servan-Schreiber, S.: FOLEAGE: $\mathbb{F}_4$ OLE-based multi-party computation for boolean circuits. In: *Advances in Cryptology – ASIACRYPT 2024, Part VI*. pp. 69–101. Lecture Notes in Computer Science, Springer, Singapore, Singapore (Dec 7–11, 2024). https://doi.org/10.1007/978-981-96-0938-3_3
8. Bombar, M., Couteau, G., Couvreur, A., Ducros, C.: Correlated pseudorandomness from the hardness of quasi-abelian decoding. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023, Part IV*. Lecture Notes in Computer Science, vol. 14084, pp. 567–601. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38551-3_18
9. Bouillaguet, C., Delaplace, C., Hamdad, M., Vergnaud, D.: Practical cryptanalysis of pseudorandom correlation generators based on quasi-abelian syndrome decoding. *Cryptology ePrint Archive*, Paper 2025/892 (2025), <https://eprint.iacr.org/2025/892>
10. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y.: Compressing vector OLE. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) *ACM CCS 2018: 25th Conference on Computer and Communications Security*. pp. 896–912. ACM Press, Toronto, ON, Canada (Oct 15–19, 2018). <https://doi.org/10.1145/3243734.3243868>
11. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Resch, N., Scholl, P.: Correlated pseudorandomness from expand-accumulate codes. In: Dodis, Y., Shrimpton, T. (eds.) *Advances in Cryptology – CRYPTO 2022, Part II*. Lecture Notes in Computer Science, vol. 13508, pp. 603–633. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 15–18, 2022). https://doi.org/10.1007/978-3-031-15979-4_21
12. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Rindal, P., Scholl, P.: Efficient two-round OT extension and silent non-interactive secure computation. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) *ACM CCS 2019: 26th Conference on Computer and Communications Security*. pp. 291–308. ACM Press, London, UK (Nov 11–15, 2019). <https://doi.org/10.1145/3319535.3354255>
13. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Scholl, P.: Efficient pseudorandom correlation generators: Silent OT extension and more. In: Boldyreva, A., Micciancio, D. (eds.) *Advances in Cryptology – CRYPTO 2019, Part III*. Lecture Notes in Computer Science, vol. 11694, pp. 489–518. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2019). https://doi.org/10.1007/978-3-030-26954-8_16
14. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Scholl, P.: Efficient pseudorandom correlation generators from ring-LPN. In: Micciancio, D., Ristenpart, T. (eds.) *Advances in Cryptology – CRYPTO 2020, Part II*. Lecture Notes in Computer Science, vol. 12171, pp. 387–416. Springer, Cham, Switzerland, Santa

- Barbara, CA, USA (Aug 17–21, 2020). https://doi.org/10.1007/978-3-030-56880-1_14
15. Brakerski, Z., Branco, P., Döttling, N., Pu, S.: Batch-OT with optimal rate. In: Dunkelman, O., Dziembowski, S. (eds.) *Advances in Cryptology – EUROCRYPT 2022, Part II. Lecture Notes in Computer Science*, vol. 13276, pp. 157–186. Springer, Cham, Switzerland, Trondheim, Norway (May 30 – Jun 3, 2022). https://doi.org/10.1007/978-3-031-07085-3_6
 16. Briaud, P., Øygarden, M.: A new algebraic approach to the regular syndrome decoding problem and implications for PCG constructions. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology – EUROCRYPT 2023, Part V. Lecture Notes in Computer Science*, vol. 14008, pp. 391–422. Springer, Cham, Switzerland, Lyon, France (Apr 23–27, 2023). https://doi.org/10.1007/978-3-031-30589-4_14
 17. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: 42nd Annual Symposium on Foundations of Computer Science. pp. 136–145. IEEE Computer Society Press, Las Vegas, NV, USA (Oct 14–17, 2001). <https://doi.org/10.1109/SFCS.2001.959888>
 18. Carrier, K., Debris-Alazard, T., Meyer-Hilfiger, C., Tillich, J.P.: Statistical decoding 2.0: Reducing decoding to LPN. In: Agrawal, S., Lin, D. (eds.) *Advances in Cryptology – ASIACRYPT 2022, Part IV. Lecture Notes in Computer Science*, vol. 13794, pp. 477–507. Springer, Cham, Switzerland, Taipei, Taiwan (Dec 5–9, 2022). https://doi.org/10.1007/978-3-031-22972-5_17
 19. Couteau, G.: New protocols for secure equality test and comparison. In: Preneel, B., Vercauteren, F. (eds.) *ACNS 18: 16th International Conference on Applied Cryptography and Network Security. Lecture Notes in Computer Science*, vol. 10892, pp. 303–320. Springer, Cham, Switzerland, Leuven, Belgium (Jul 2–4, 2018). https://doi.org/10.1007/978-3-319-93387-0_16
 20. Couteau, G., Rindal, P., Raghuraman, S.: Silver: Silent VOLE and oblivious transfer from hardness of decoding structured LDPC codes. In: Malkin, T., Peikert, C. (eds.) *Advances in Cryptology – CRYPTO 2021, Part III. Lecture Notes in Computer Science*, vol. 12827, pp. 502–534. Springer, Cham, Switzerland, Virtual Event (Aug 16–20, 2021). https://doi.org/10.1007/978-3-030-84252-9_17
 21. Damgård, I., Pastro, V., Smart, N.P., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. In: Safavi-Naini, R., Canetti, R. (eds.) *Advances in Cryptology – CRYPTO 2012. Lecture Notes in Computer Science*, vol. 7417, pp. 643–662. Springer, Berlin, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 19–23, 2012). https://doi.org/10.1007/978-3-642-32009-5_38
 22. Dao, Q., Ishai, Y., Jain, A., Lin, H.: Multi-party homomorphic secret sharing and sublinear MPC from sparse LPN. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023, Part II. Lecture Notes in Computer Science*, vol. 14082, pp. 315–348. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38545-2_11
 23. Dodis, Y., Halevi, S., Rothblum, R.D., Wichs, D.: Spooky encryption and its applications. In: Robshaw, M., Katz, J. (eds.) *Advances in Cryptology – CRYPTO 2016, Part III. Lecture Notes in Computer Science*, vol. 9816, pp. 93–122. Springer, Berlin, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 14–18, 2016). https://doi.org/10.1007/978-3-662-53015-3_4
 24. Döttling, N., Garg, S., Ishai, Y., Malavolta, G., Mour, T., Ostrovsky, R.: Trapdoor hash functions and their applications. In: Boldyreva, A., Micciancio, D. (eds.) *Advances in Cryptology – CRYPTO 2019, Part III. Lecture Notes in Computer*

- Science, vol. 11694, pp. 3–32. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2019). https://doi.org/10.1007/978-3-030-26954-8_1
25. Dumer, I.: On minimum distance decoding of linear codes
26. ElGamal, T.: A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory* **31**(4), 469–472 (1985). <https://doi.org/10.1109/TIT.1985.1057074>
27. Esser, A., Kübler, R., May, A.: LPN decoded. In: Katz, J., Shacham, H. (eds.) *Advances in Cryptology – CRYPTO 2017, Part II. Lecture Notes in Computer Science*, vol. 10402, pp. 486–514. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2017). https://doi.org/10.1007/978-3-319-63715-0_17
28. Fossorier, M.P.C., Kobara, K., Imai, H.: Modeling bit flipping decoding based on nonorthogonal check sums with application to iterative decoding attack of mceliece cryptosystem. *IEEE Transactions on Information Theory* **53**(1), 402–411 (2007). <https://doi.org/10.1109/TIT.2006.887515>
29. Genise, N., Micciancio, D., Peikert, C., Walter, M.: Improved discrete gaussian and subgaussian analysis for lattice cryptography. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) *PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part I. Lecture Notes in Computer Science*, vol. 12110, pp. 623–651. Springer, Cham, Switzerland, Edinburgh, UK (May 4–7, 2020). https://doi.org/10.1007/978-3-030-45374-9_21
30. Goldreich, O., Goldwasser, S., Micali, S.: How to construct random functions (extended abstract). In: *25th Annual Symposium on Foundations of Computer Science*. pp. 464–479. IEEE Computer Society Press, Singer Island, Florida (Oct 24–26, 1984). <https://doi.org/10.1109/SFCS.1984.715949>
31. Goldreich, O., Micali, S., Wigderson, A.: How to play any mental game or A completeness theorem for protocols with honest majority. In: Aho, A. (ed.) *19th Annual ACM Symposium on Theory of Computing*. pp. 218–229. ACM Press, New York City, NY, USA (May 25–27, 1987). <https://doi.org/10.1145/28395.28420>
32. Goyal, V., Li, J., Misra, A.K., Ostrovsky, R., Song, Y., Weng, C.: Dishonest majority constant-round MPC with linear communication from DDH. In: *Advances in Cryptology – ASIACRYPT 2024, Part VI*. pp. 167–199. *Lecture Notes in Computer Science*, Springer, Singapore, Singapore (Dec 7–11, 2024). https://doi.org/10.1007/978-981-96-0938-3_6
33. Ishai, Y., Kilian, J., Nissim, K., Petrank, E.: Extending oblivious transfers efficiently. In: Boneh, D. (ed.) *Advances in Cryptology – CRYPTO 2003. Lecture Notes in Computer Science*, vol. 2729, pp. 145–161. Springer, Berlin, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 17–21, 2003). https://doi.org/10.1007/978-3-540-45146-4_9
34. Keller, M., Orsini, E., Scholl, P.: MASCOT: Faster malicious arithmetic secure computation with oblivious transfer. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) *ACM CCS 2016: 23rd Conference on Computer and Communications Security*. pp. 830–842. ACM Press, Vienna, Austria (Oct 24–28, 2016). <https://doi.org/10.1145/2976749.2978357>
35. Keller, M., Pastro, V., Rotaru, D.: Overdrive: Making SPDZ great again. In: Nielsen, J.B., Rijmen, V. (eds.) *Advances in Cryptology – EUROCRYPT 2018, Part III. Lecture Notes in Computer Science*, vol. 10822, pp. 158–189. Springer, Cham, Switzerland, Tel Aviv, Israel (Apr 29 – May 3, 2018). https://doi.org/10.1007/978-3-319-78372-7_6
36. Li, Z., Xing, C., Yao, Y., Yuan, C.: Efficient pseudorandom correlation generators for any finite field. In: Fehr, S., Fouque, P. (eds.) *Advances in Cryptology -*

- EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part V. Lecture Notes in Computer Science, vol. 15605, pp. 145–175. Springer (2025). https://doi.org/10.1007/978-3-031-91092-0_6, https://doi.org/10.1007/978-3-031-91092-0_6
37. Liu, H., Wang, X., Yang, K., Yu, Y.: The hardness of LPN over any integer ring and field for PCG applications. In: Joye, M., Leander, G. (eds.) *Advances in Cryptology – EUROCRYPT 2024*, Part VI. Lecture Notes in Computer Science, vol. 14656, pp. 149–179. Springer, Cham, Switzerland, Zurich, Switzerland (May 26–30, 2024). https://doi.org/10.1007/978-3-031-58751-1_6
 38. May, A., Meurer, A., Thomae, E.: Decoding random linear codes in $\tilde{O}(2^{0.054n})$. In: Lee, D.H., Wang, X. (eds.) *Advances in Cryptology – ASIACRYPT 2011*. Lecture Notes in Computer Science, vol. 7073, pp. 107–124. Springer, Berlin, Heidelberg, Germany, Seoul, South Korea (Dec 4–8, 2011). https://doi.org/10.1007/978-3-642-25385-0_6
 39. Overbeck, R.: Statistical decoding revisited. In: Batten, L.M., Safavi-Naini, R. (eds.) *ACISP 06: 11th Australasian Conference on Information Security and Privacy*. Lecture Notes in Computer Science, vol. 4058, pp. 283–294. Springer, Berlin, Heidelberg, Germany, Melbourne, Australia (Jul 3–5, 2006). https://doi.org/10.1007/11780656_24
 40. Peter Rindal, L.R.: libOTe: an efficient, portable, and easy to use Oblivious Transfer Library. <https://github.com/osu-crypto/libOTe>
 41. Prange, E.: The use of information sets in decoding cyclic codes. *IRE Transactions on Information Theory* **8**(5), 5–9 (1962). <https://doi.org/10.1109/TIT.1962.1057777>
 42. Rabin, M.O.: How to exchange secrets with oblivious transfer. *Cryptology ePrint Archive*, Report 2005/187 (2005), <https://eprint.iacr.org/2005/187>
 43. Raghuraman, S., Rindal, P., Tanguy, T.: Expand-convolute codes for pseudorandom correlation generators from LPN. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023*, Part IV. Lecture Notes in Computer Science, vol. 14084, pp. 602–632. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38551-3_19
 44. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: Gabow, H.N., Fagin, R. (eds.) *37th Annual ACM Symposium on Theory of Computing*. pp. 84–93. ACM Press, Baltimore, MA, USA (May 22–24, 2005). <https://doi.org/10.1145/1060590.1060603>
 45. Rindal, P.: CryptoTools: portable C++14 library for building cryptographic protocols. <https://github.com/ladnir/cryptoTools>
 46. Schoppmann, P., Gascón, A., Reichert, L., Raykova, M.: Distributed vector-OLE: Improved constructions and implementation. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) *ACM CCS 2019: 26th Conference on Computer and Communications Security*. pp. 1055–1072. ACM Press, London, UK (Nov 11–15, 2019). <https://doi.org/10.1145/3319535.3363228>
 47. Stern, J.: A method for finding codewords of small weight. In: *Proceedings of the 3rd International Colloquium on Coding Theory and Applications*. p. 106–113. Springer-Verlag, Berlin, Heidelberg (1988)
 48. Wang, X., Ranellucci, S., Katz, J.: Authenticated garbling and efficient maliciously secure two-party computation. In: Thuraishingham, B.M., Evans, D., Malkin, T., Xu, D. (eds.) *ACM CCS 2017: 24th Conference on Computer and Communications Security*. pp. 21–37. ACM Press, Dallas, TX, USA (Oct 31 – Nov 2, 2017). <https://doi.org/10.1145/3133956.3134053>

49. Wang, X., Ranellucci, S., Katz, J.: Global-scale secure multiparty computation. In: Thuraisingham, B.M., Evans, D., Malkin, T., Xu, D. (eds.) ACM CCS 2017: 24th Conference on Computer and Communications Security. pp. 39–56. ACM Press, Dallas, TX, USA (Oct 31 – Nov 2, 2017). <https://doi.org/10.1145/3133956.3133979>
50. Yao, A.C.C.: How to generate and exchange secrets (extended abstract). In: 27th Annual Symposium on Foundations of Computer Science. pp. 162–167. IEEE Computer Society Press, Toronto, Ontario, Canada (Oct 27–29, 1986). <https://doi.org/10.1109/SFCS.1986.25>

A Multiparty Beaver Triples

PCG for two-party programmable OLE implies PCG for multiparty programmable Beaver triples with a multiplicative overhead of $2(n-1)$ in the seed size and expansion cost per party [13]. The distributed seed generation phase requires $n \cdot (n-1)$ executions of the two-party seed generation protocol. For completeness, we define the ideal functionality for multiparty Beaver triples in Figure 6 and present the protocol in Figure 7.

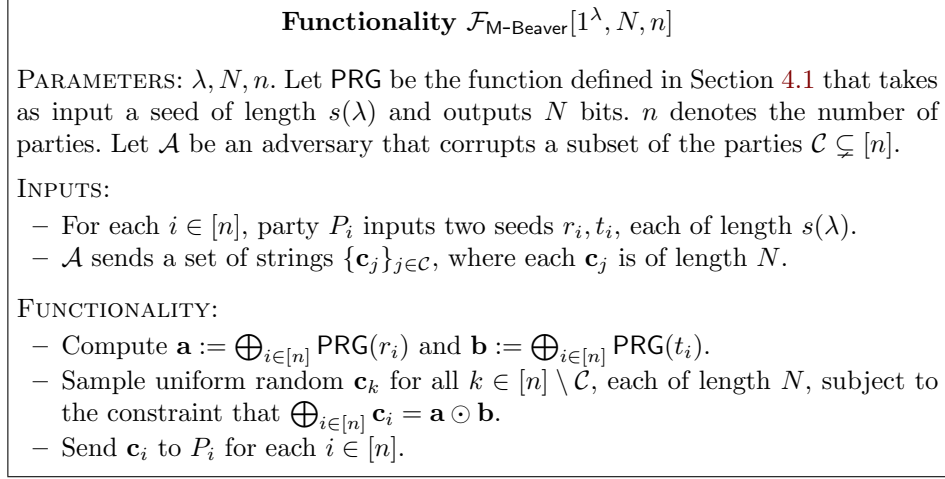


Fig. 6: Functionality for Multiparty Beaver Triples.

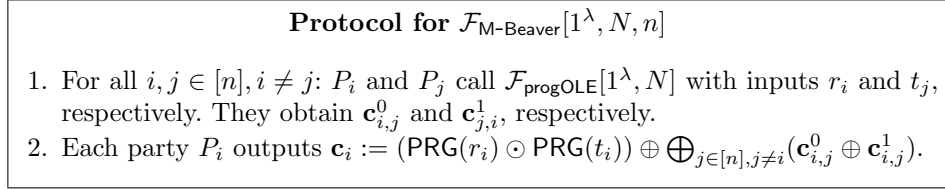


Fig. 7: Protocol for Multiparty Beaver Triples.